



US 20250278887A1

(19) **United States**

(12) **Patent Application Publication**

Story et al.

(10) **Pub. No.: US 2025/0278887 A1**

(43) **Pub. Date:**

Sep. 4, 2025

(54) **PERFORMANCE OF RAY-TRACED SHADOW CREATION WITHIN A SCENE**

(71) Applicant: **NVIDIA Corporation**, Santa Clara, CA (US)

(72) Inventors: **Jon Story**, Grafrath (DE); **Holger Heinrich Gruen**, Bavaria (DE)

(21) Appl. No.: **19/212,469**

(22) Filed: **May 19, 2025**

Related U.S. Application Data

(63) Continuation of application No. 18/383,824, filed on Oct. 25, 2023, now Pat. No. 12,333,642, which is a continuation of application No. 17/231,979, filed on Apr. 15, 2021, now Pat. No. 11,847,733, which is a continuation of application No. 16/522,539, filed on Jul. 25, 2019, now Pat. No. 11,004,254.

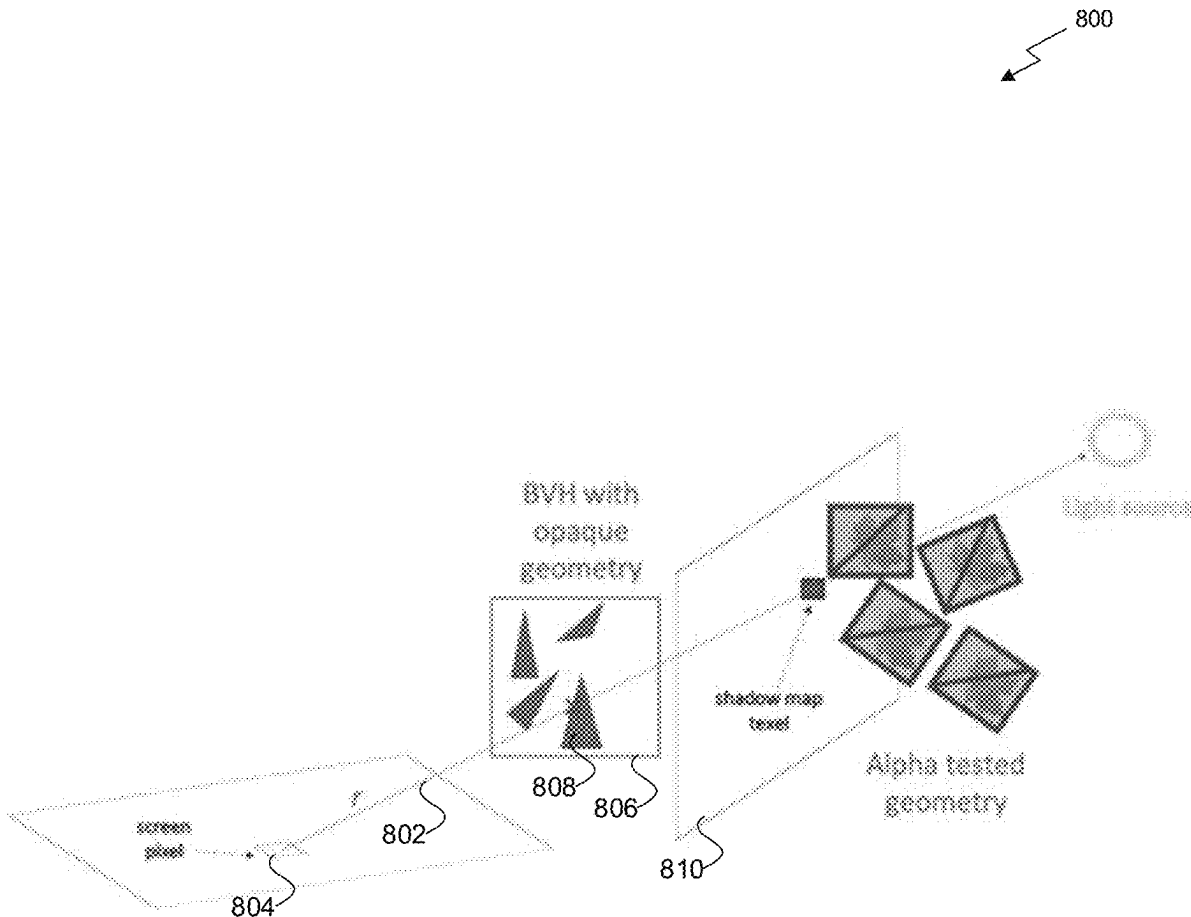
Publication Classification

(51) **Int. Cl.**
G06T 15/06 (2011.01)
G06T 5/70 (2024.01)
G06T 15/00 (2011.01)

G06T 15/04 (2011.01)
G06T 15/40 (2011.01)
G06T 15/60 (2006.01)
(52) **U.S. Cl.**
CPC **G06T 15/06** (2013.01); **G06T 5/70** (2024.01); **G06T 15/005** (2013.01); **G06T 15/405** (2013.01); **G06T 15/60** (2013.01); **G06T 2215/12** (2013.01)

(57) **ABSTRACT**

A ray (e.g., a traced path of light, etc.) is generated from an originating pixel within a scene being rendered. Additionally, one or more shadow map lookups are performed for the originating pixel to estimate an intersection of the ray with alpha-tested geometry within the scene. A shadow map stores the distance of geometry as seen from the point of view of the light, and alpha-tested geometry includes objects within the scene being rendered that have a determined texture and opacity. Further, the one or more shadow map lookups are performed to determine a visibility value for the pixel (e.g., that identifies whether the originating pixel is in a shadow) and a distance value for the pixel (e.g., that identifies how far the pixel is from the light). Further still, the visibility value and the distance value for the pixel are passed to a denoiser.



100
↙

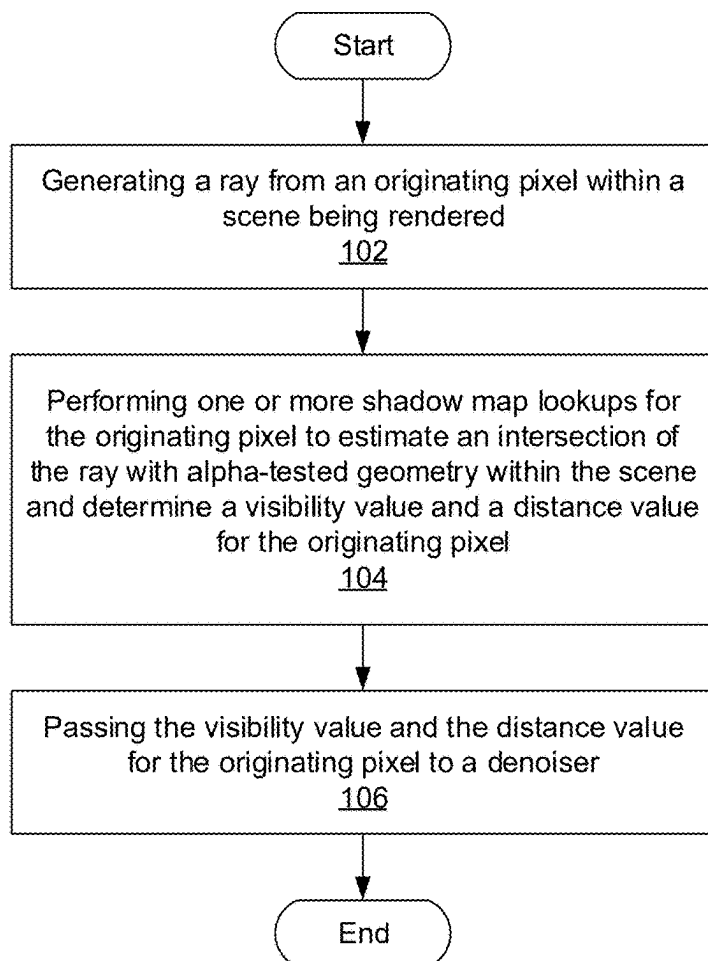


Fig. 1

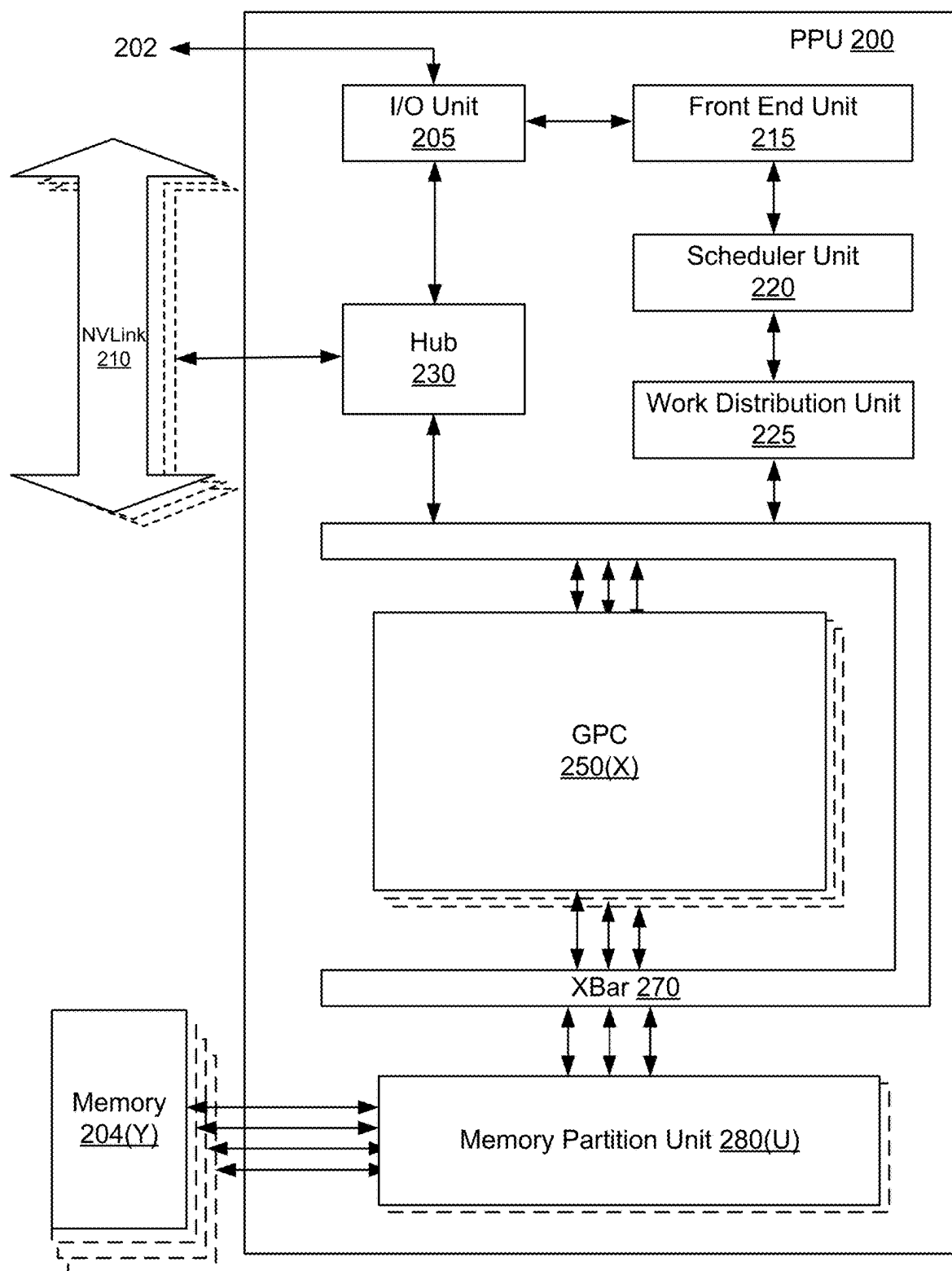


Fig. 2

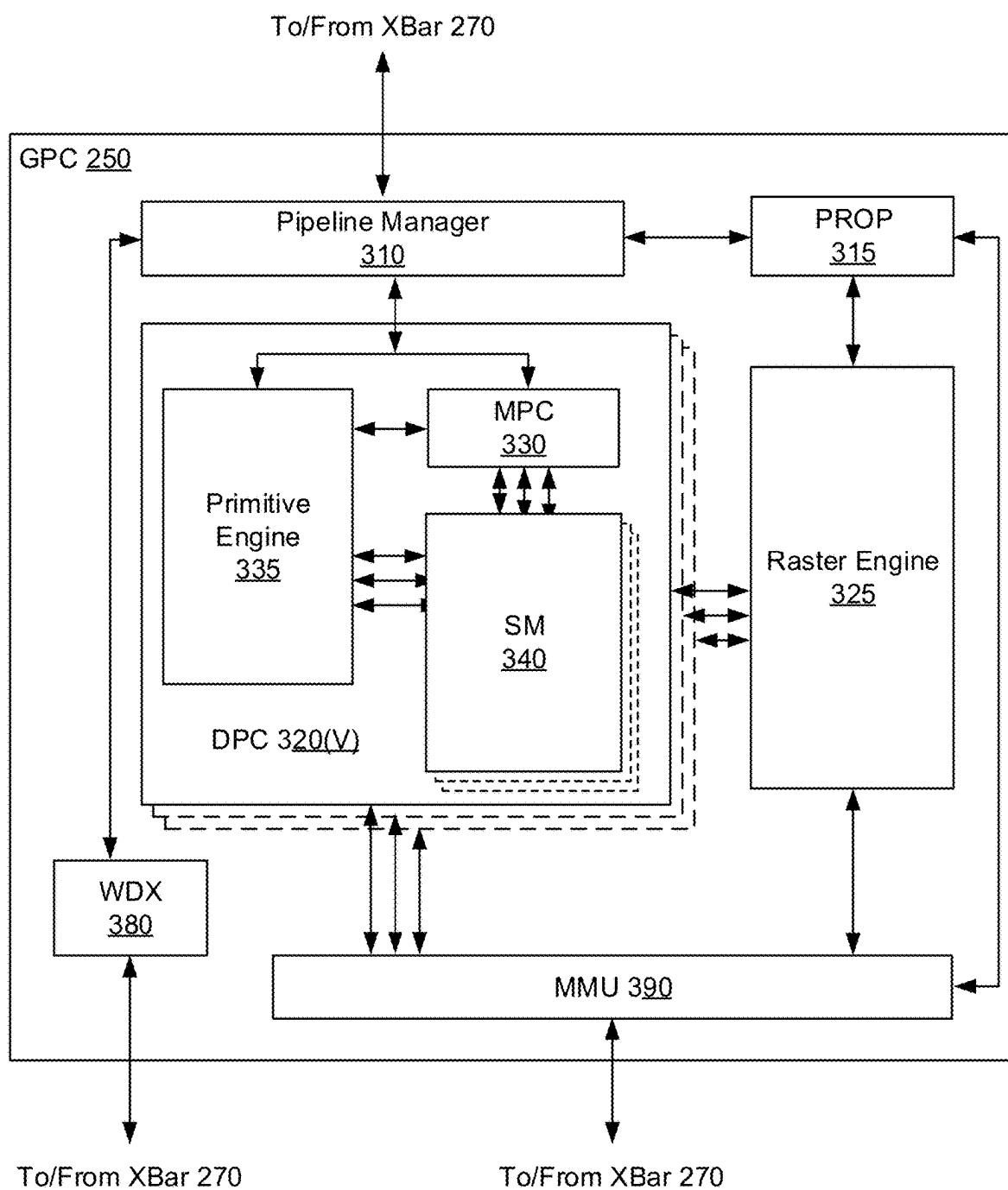
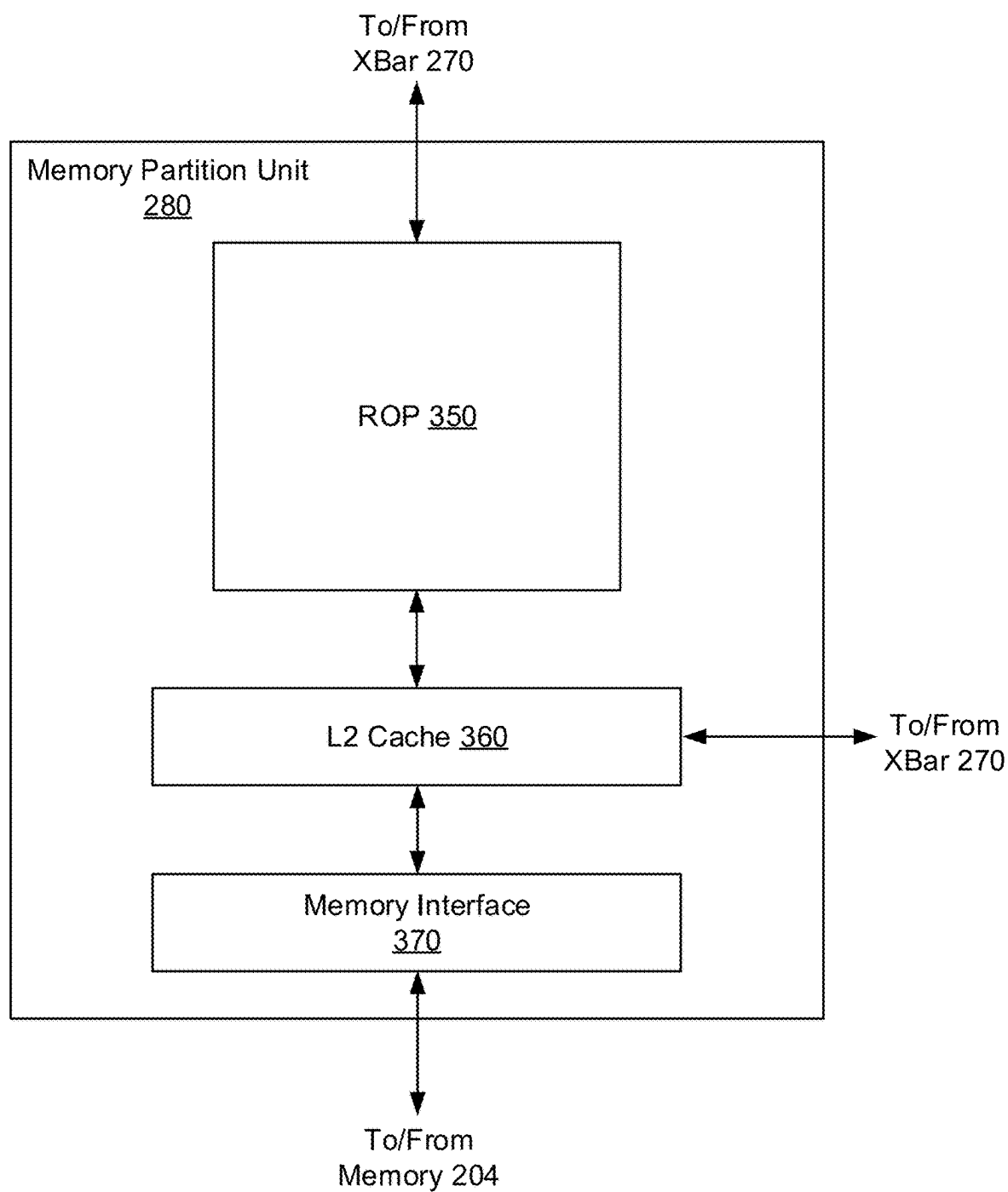


Fig. 3A

***Fig. 3B***

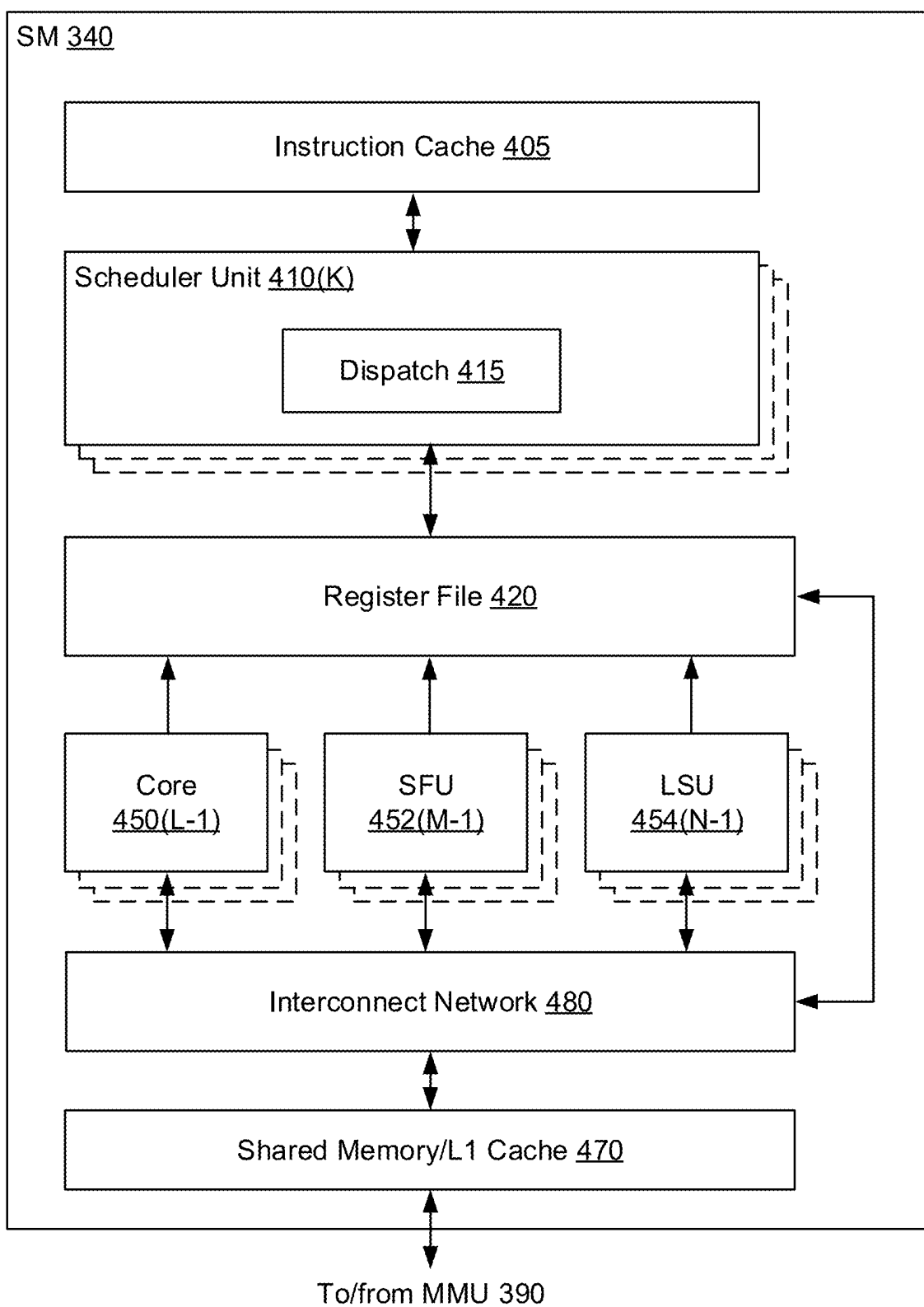


Fig. 4A

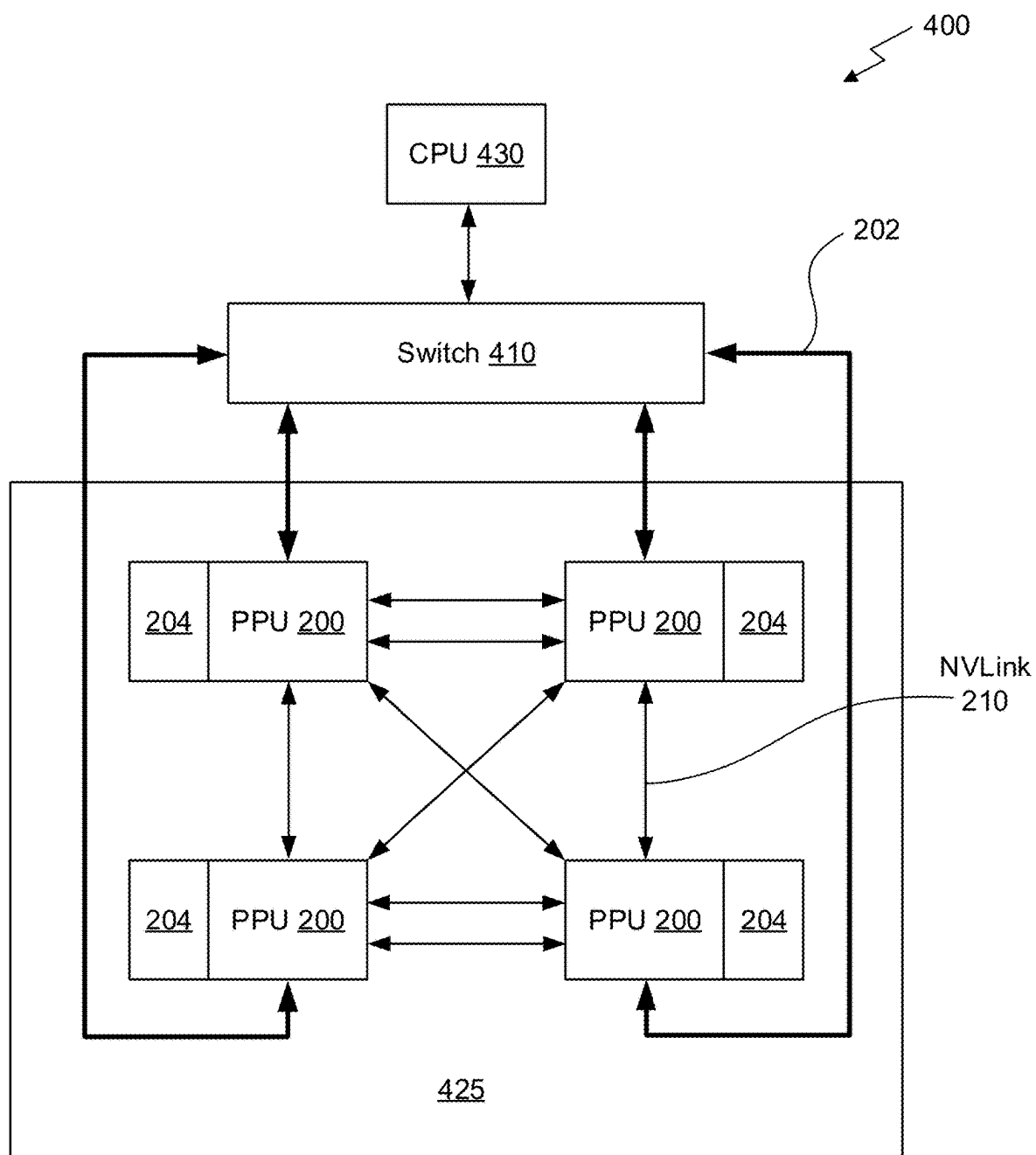


Fig. 4B

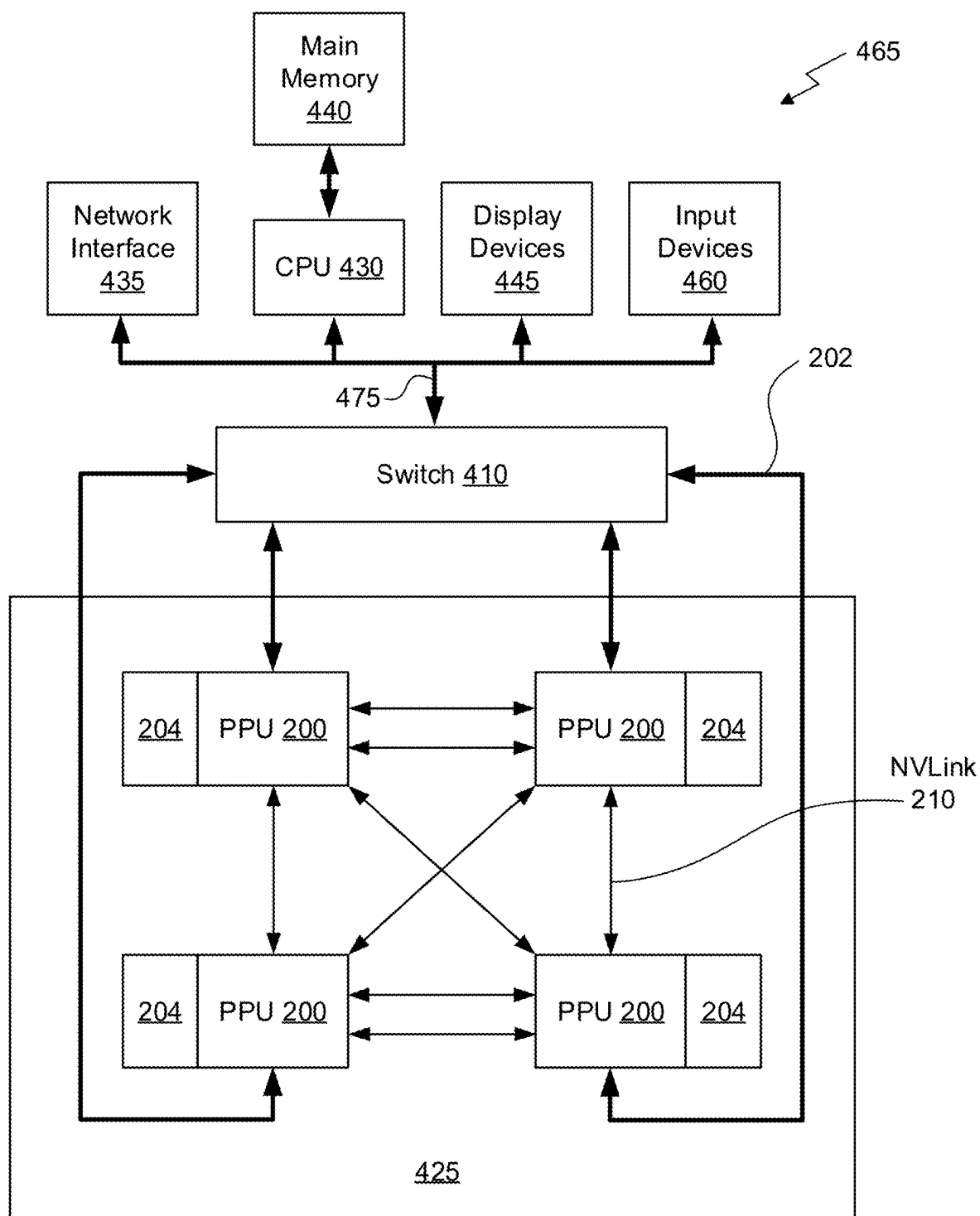
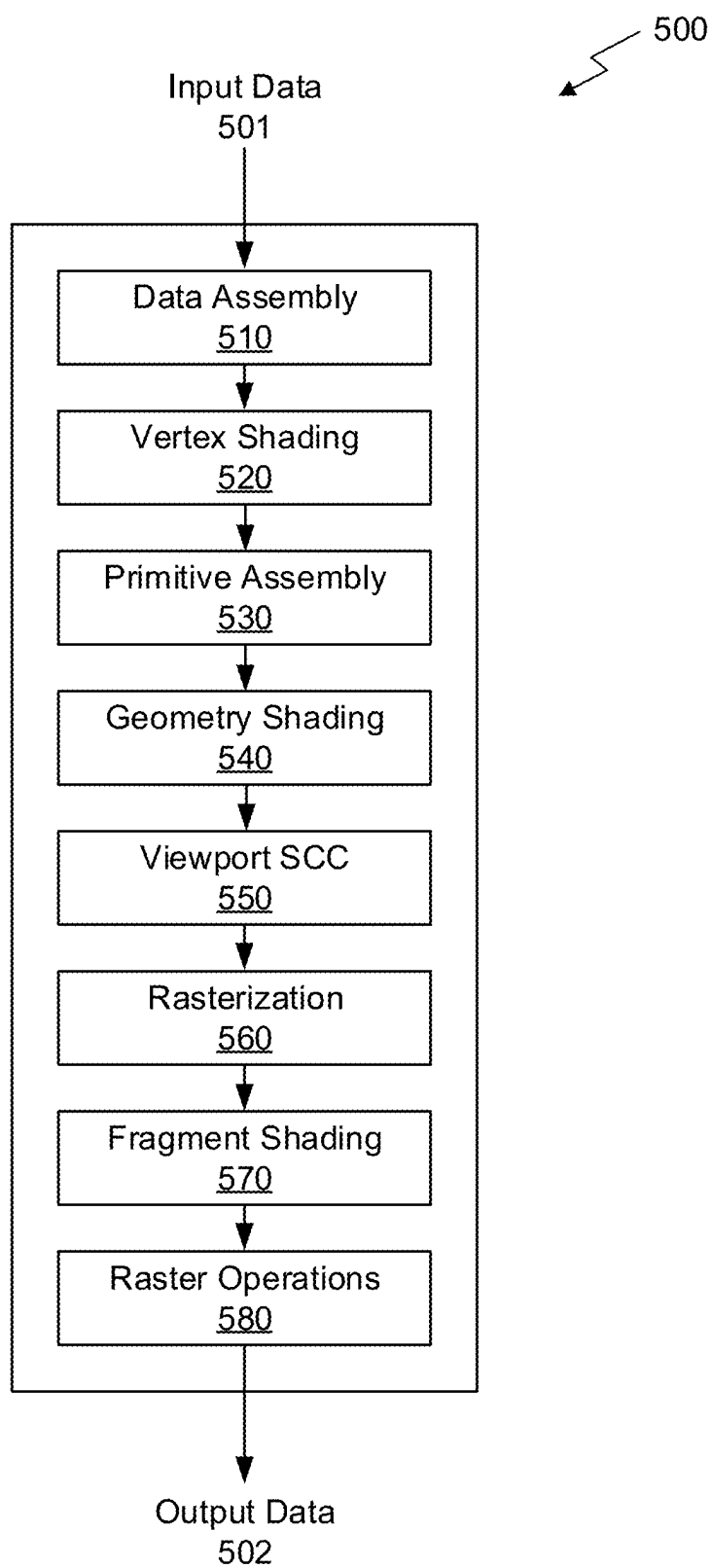


Fig. 4C

*Fig. 5*

600
↙

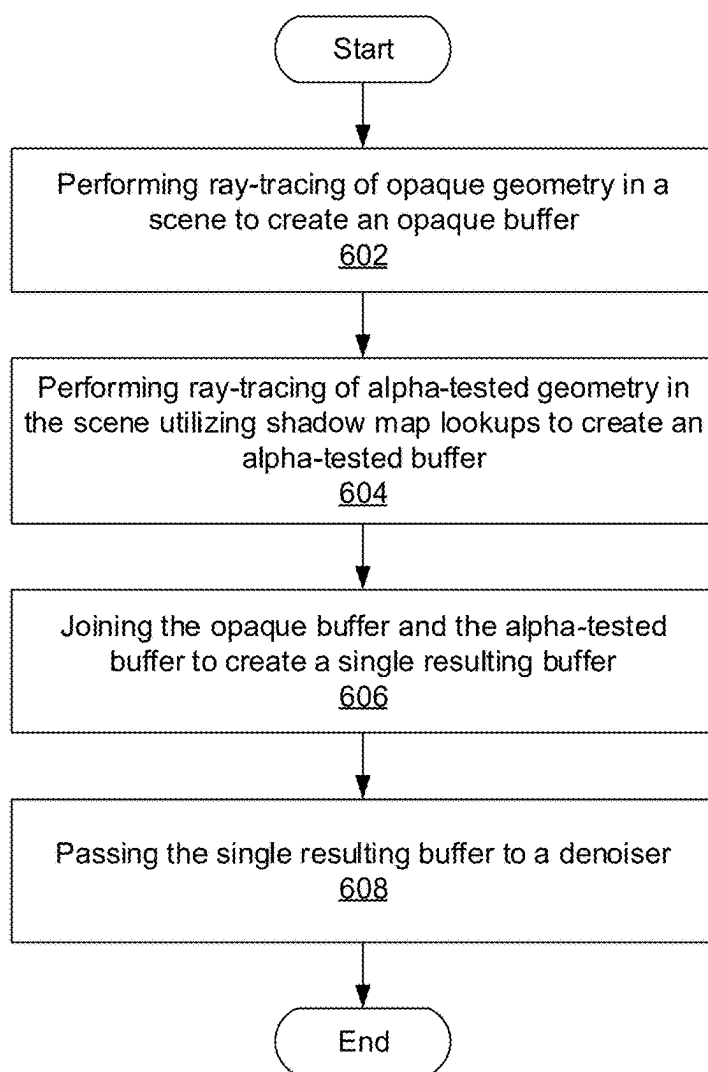
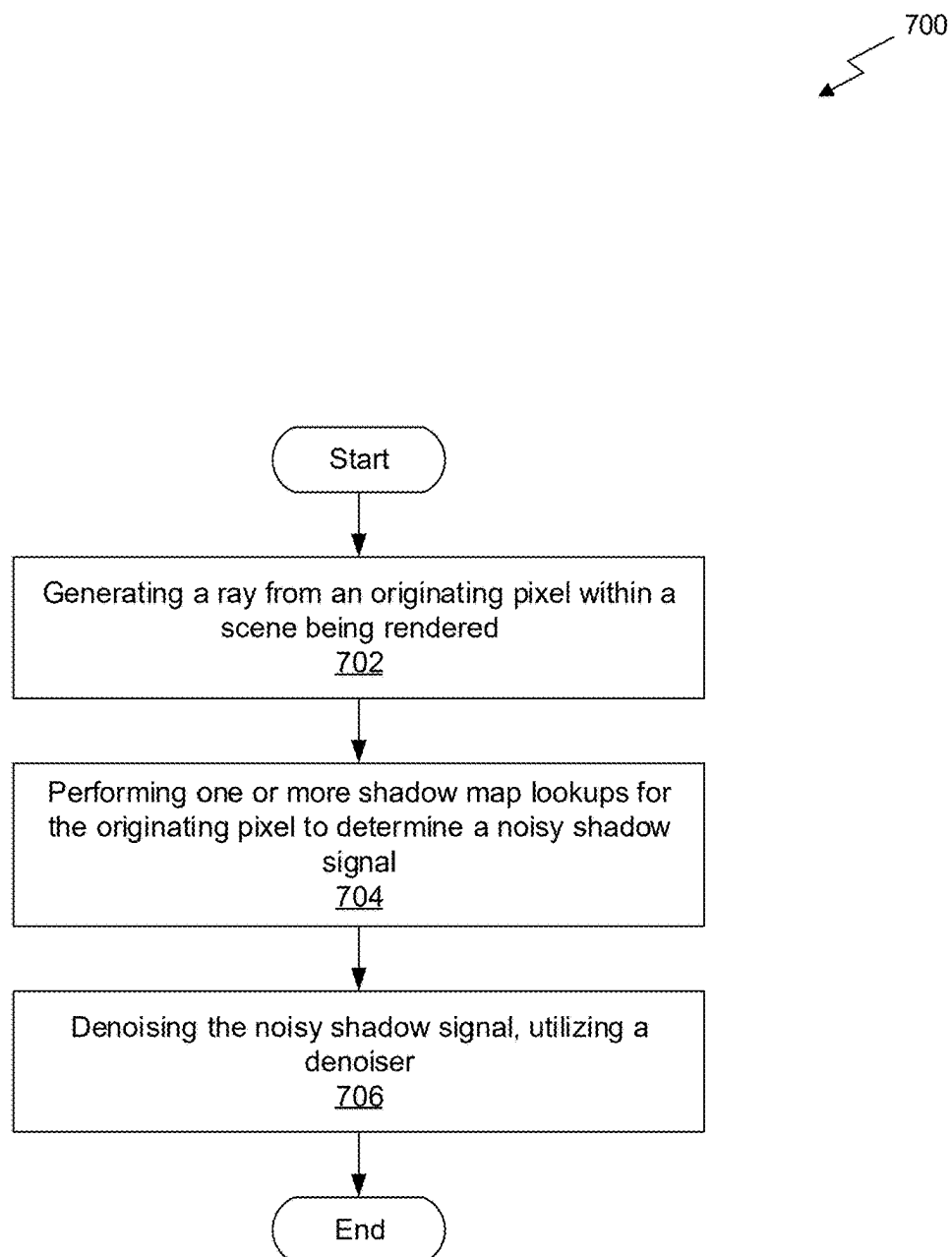


Fig. 6

**Fig. 7**

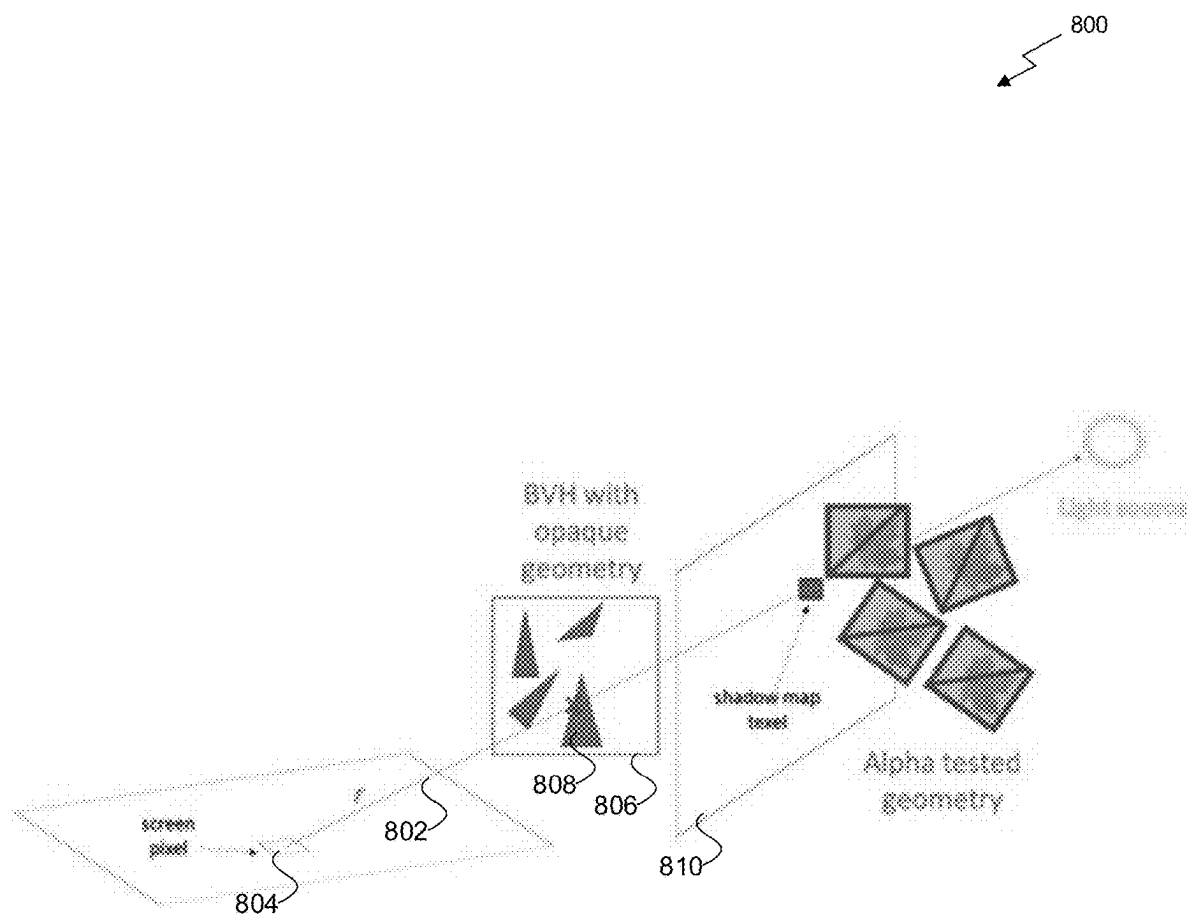


Fig. 8

PERFORMANCE OF RAY-TRACED SHADOW CREATION WITHIN A SCENE

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is a continuation application of U.S. patent application Ser. No. 18/383,824, filed Oct. 25, 2023, which in turn is a continuation of U.S. patent application Ser. No. 17/231,979, filed Apr. 15, 2021, now U.S. Pat. No. 11,847,733 granted Dec. 19, 2023, which in turn is a continuation of U.S. patent application Ser. No. 16/522,539, filed Jul. 25, 2019, now U.S. Pat. No. 11,004,254 granted May 11, 2021, all of which are hereby incorporated by reference in their entirety.

FIELD OF THE INVENTION

[0002] The present invention relates to image rendering, and more particularly to rendering shadows within a scene.

BACKGROUND

[0003] Ray-tracing shadows within a scene typically aims at shooting only a small number of noisy/jittered shadow rays. This results in a noisy shadow signal. In order to achieve realistic shadows within the same scene that result in high-quality penumbras (partially shaded outer regions of shadows cast by an object, where these pixels can only see a fraction of the area of the light source), this noisy signal is smoothed by an algorithm called a 'denoiser'. However, current methods for ray-tracing shadows of alpha-tested geometry are very resource-expensive. This cost is amplified when a large quantity of alpha-tested geometry is located within a scene (e.g., as leaves within trees or other foliage, etc.).

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] FIG. 1 illustrates a flowchart of a method for improving performance of ray-traced shadow creation within a scene, in accordance with an embodiment.

[0005] FIG. 2 illustrates a parallel processing unit, in accordance with an embodiment.

[0006] FIG. 3A illustrates a general processing cluster within the parallel processing unit of FIG. 2, in accordance with an embodiment.

[0007] FIG. 3B illustrates a memory partition unit of the parallel processing unit of FIG. 2, in accordance with an embodiment.

[0008] FIG. 4A illustrates the streaming multi-processor of FIG. 3A, in accordance with an embodiment.

[0009] FIG. 4B is a conceptual diagram of a processing system implemented using the PPU of FIG. 2, in accordance with an embodiment.

[0010] FIG. 4C illustrates an exemplary system in which the various architecture and/or functionality of the various previous embodiments may be implemented.

[0011] FIG. 5 is a conceptual diagram of a graphics processing pipeline implemented by the PPU of FIG. 2, in accordance with an embodiment.

[0012] FIG. 6 illustrates a flowchart of a method for joining opaque and alpha-tested buffers, in accordance with one embodiment.

[0013] FIG. 7 illustrates a flowchart of a method for implementing denoised shadows using only shadow map lookups, in accordance with one embodiment.

[0014] FIG. 8 illustrates an exemplary determination of hybrid denoised shadows, in accordance with one embodiment.

DETAILED DESCRIPTION

[0015] In computer graphics, ray-tracing is often used to generate an image by tracing paths of light within an image plane and determining effects of intersections of those paths of light with objects (e.g., geometric shapes, etc.) within a scene. For example, ray-tracing may be used to create realistic shadows within the scene. Some of the objects within the scene are known as alpha-tested geometry, which have a texture mapped to them, where the texture has an associated alpha channel that measures a transparency of the texture.

[0016] Currently, when alpha-tested geometry is encountered during the ray-tracing of the scene, several time-consuming operations need to be performed to determine accurate shadows. As a result, scenes with a large quantity of alpha-tested geometry are time and resource-intensive to render. To address this, one or more shadow map lookups are performed during ray-tracing to estimate an intersection of a ray with alpha-tested geometry within the scene. A shadow map stores the distance of geometry as seen from the point of view of a light source. The shadow map lookups are also used to determine a visibility value for the pixel (that identifies whether the originating pixel is in a shadow) and a distance value for the pixel (that identifies how far the pixel is from the light). These values are then passed to a denoiser, which may utilize the values to smooth out a resulting image during the rendering process. A denoiser operates by removing noise (such as unrealistic visual artifacts) from a displayed scene. A denoiser applied to the rendering of shadows within a scene may use the visibility and distance values to determine how much blurring to apply to the shadows within the scene to make them as realistic as possible. For example, pixels with small distance values may indicate hard shadows close to a light source that are rendered by the denoiser with little blurring. Likewise, pixels with high distance values may indicate softer shadows further from a light source that are rendered by the denoiser with more blurring than hard shadows. This approach improves a realism of a resulting scene as well as a performance of ray-tracing with alpha-tested geometry.

[0017] FIG. 1 illustrates a flowchart of a method 100 for improving performance of ray-traced shadow creation within a scene, in accordance with an embodiment. Although method 100 is described in the context of a processing unit, the method 100 may also be performed by a program, custom circuitry, or by a combination of custom circuitry and a program. For example, the method 100 may be executed by a GPU (graphics processing unit), CPU (central processing unit), or any processor capable of performing parallel path space filtering by hashing. Furthermore, persons of ordinary skill in the art will understand that any system that performs method 100 is within the scope and spirit of embodiments of the present invention.

[0018] As shown in operation 102, a shadow-ray is generated from an originating pixel towards the light within a scene being rendered to test if the light reaches the originating pixel. The geometry in this scene may include any geometric shape (e.g., a triangle, etc.). In one embodiment, the ray may be generated as part of a ray-tracing operation (e.g., a raytracing of shadows of geometry within the scene,

etc.). Tracing rays for each on-screen pixel result in a screen-sized buffer that contains a pair of values: 1) a visibility value, and 2) a hitT/distance value. If the visibility value is 0, the hitT/distance value stores the distance between the originating pixel and the piece of geometry that stops the shadow-ray from reaching the light source.

[0019] Additionally, as shown in operation **104**, an intersection of the ray with alpha-tested geometry is identified within the scene. In one embodiment, the intersection may be identified during a ray-tracing operation. For example, the ray may originate from a predetermined originating pixel in the scene and may be traced from the originating pixel to a light source. In another embodiment, the ray may intersect geometry within the scene during the tracing.

[0020] In another embodiment, only alpha-tested geometry within the scene gets drawn into an alpha-only shadow map. A shadow map stores the distance of geometry as seen from the point of view of the light, the alpha-only shadow map only contains depth values from geometry that passes the alpha-test. For example, for alpha-tested geometry, a lookup into an alpha texture indicates whether a predetermined portion of the geometry is opaque or transparent and if any output is written to the alpha-only shadow map.

[0021] Further still, as shown in operation **106**, one or more shadow map lookups are performed for the originating pixel of a shadow-ray to determine a visibility value and a distance/hit value for the originating pixel from data that is stored in the alpha-only shadow map. In order to do a shadow map lookup the position of the originating pixel needs to be projected to the two-dimensional (2D) coordinate system of the shadow map.

[0022] Also, in one embodiment, a location of the intersection of a shadow-ray with the pixels in the alpha-only shadow map may be estimated from data within the alpha-only shadow map. For example, the shadow map texture may be sampled to determine a location of this intersection within the shadow map. In another embodiment, the visibility value for the originating pixel may be determined based on one or more depth values at or around the estimated intersection location within the shadow map.

[0023] For example, in shadow mapping, the depth values as seen from the point of view of the light, are rendered into a shadow map may. These depth values can then be compared with a depth value of the current pixel. If the depth value of the current pixel is larger (e.g., further away) than the depth value stored in the shadow map, the pixel is determined to be in a shadow (and is therefore given a visibility value of zero). If the depth value of the current pixel is smaller (e.g., closer) than the depth value stored in the shadow map, the pixel is determined to be light/not in a shadow (and is therefore given a visibility value of one).

[0024] In addition, this hitT/distance value for a shadow-ray may be determined by identifying a depth value for an estimated intersection of the ray with the shadow map and determining a difference in distance between the originating pixel location and the depth value.

[0025] Further still, as shown in operation **108**, the visibility value and the hitT/distance value for the originating pixel are passed to a denoiser. In one embodiment, the visibility value and the hitT/distance value may be stored in a buffer. For example, the visibility value and the distance value may be stored with a plurality of other visibility and distance values for other originating pixels in the buffer. In one example, one buffer may store visibility and distance

values for originating pixels within the scene from rays that intersect with occluding geometry that is purely opaque (e.g. not alpha-tested). In another example, a second buffer may store visibility and distance values for originating pixels within the scene from rays that intersect with only alpha-tested occluding geometry. In yet another embodiment, the visibility and distance values for alpha-tested occluding geometry may be determined solely from data of an alpha-only shadow map as described above.

[0026] Also, in one embodiment, the alpha-tested hitT/visibility buffer may be joined with the opaque hitT/visibility buffer. For example, the opaque buffer may store visibility and distance values for other originating pixels within the scene having rays that intersect with occluding geometry that is not alpha-tested.

[0027] In addition, in one embodiment, a min operation may be performed between the alpha-tested hitT/visibility buffer and the opaque hitT/visibility buffer to obtain a single resulting hitT/visibility buffer that is sent to a denoiser. For example, the single resulting buffer may represent all pixels within the scene (e.g., a screen size buffer, etc.). In another embodiment, the denoiser may utilize received visibility values and distance values to smooth out a resulting image during the rendering process. For example, pixels with low distance values may indicate hard shadows that are rendered with little blurring, while pixels with high distance values may indicate softer shadows that are blurred more than hard shadows.

[0028] In this way, ray-traced opaque results may be combined with alpha-tested results and may be passed together through the same denoiser. This may result in the same denoising being applied to both results, which may produce a seamless result with matching penumbra sizes for opaque and alpha-tested geometry, and may create smooth, realistic shadows within a scene. Additionally, rendering alpha-tested geometry may be processed in a more efficient manner when compared to traditional raytracing approaches (e.g. alpha-tested rays), which may reduce an amount of processing and storage resources utilized by hardware performing such rendering, thereby improving performance of such hardware.

[0029] More specifically, scenes in games, virtual reality, augmented reality, and environment simulation often include significant amounts of translucent foliage (e.g., leaves from trees and bushes). This foliage is often included as alpha-tested geometry within the scene. When ray-tracing of shadows is performed within these scenes using alpha-tested shadow map lookups, such ray-tracing may be simplified, which may result in more efficient rendering operations that can be performed faster with less processor usage. This increased performance may support an increase in an associated frame rate, which may improve a realism of games, virtual reality, augmented reality, and environment simulation.

[0030] Additionally, the above alpha-tested geometry results can be passed to the denoiser together with geometry results that are not alpha-tested (e.g., results for solid, opaque objects within the scene). Denoising both of these geometry results together results in similar denoising being applied to both, which provides consistent shadows and penumbras (partially shaded outer regions of shadows) throughout the scene, which makes the scene look more

realistic. This also improves a realism of games, virtual reality, augmented reality, and environment simulation utilizing the above technology.

[0031] In yet another embodiment, shadow rays may be generated, and shadow map lookups may be performed, utilizing a parallel processing unit (PPU) such as the PPU 200 illustrated in FIG. 2.

[0032] More illustrative information will now be set forth regarding various optional architectures and features with which the foregoing framework may be implemented, per the desires of the user. It should be strongly noted that the following information is set forth for illustrative purposes and should not be construed as limiting in any manner. Any of the following features may be optionally incorporated with or without the exclusion of other features described.

Parallel Processing Architecture

[0033] FIG. 2 illustrates a parallel processing unit (PPU) 200, in accordance with an embodiment. In an embodiment, the PPU 200 is a multi-threaded processor that is implemented on one or more integrated circuit devices. The PPU 200 is a latency hiding architecture designed to process many threads in parallel. A thread (i.e., a thread of execution) is an instantiation of a set of instructions configured to be executed by the PPU 200. In an embodiment, the PPU 200 is a graphics processing unit (GPU) configured to implement a graphics rendering pipeline for processing three-dimensional (3D) graphics data in order to generate two-dimensional (2D) image data for display on a display device such as a liquid crystal display (LCD) device. In other embodiments, the PPU 200 may be utilized for performing general-purpose computations. While one exemplary parallel processor is provided herein for illustrative purposes, it should be strongly noted that such processor is set forth for illustrative purposes only, and that any processor may be employed to supplement and/or substitute for the same.

[0034] One or more PPUs 200 may be configured to accelerate thousands of High Performance Computing (HPC), data center, and machine learning applications. The PPU 200 may be configured to accelerate numerous deep learning systems and applications including autonomous vehicle platforms, deep learning, high-accuracy speech, image, and text recognition systems, intelligent video analytics, molecular simulations, drug discovery, disease diagnosis, weather forecasting, big data analytics, astronomy, molecular dynamics simulation, financial modeling, robotics, factory automation, real-time language translation, online search optimizations, and personalized user recommendations, and the like.

[0035] As shown in FIG. 2, the PPU 200 includes an Input/Output (I/O) unit 205, a front end unit 215, a scheduler unit 220, a work distribution unit 225, a hub 230, a crossbar (Xbar) 270, one or more general processing clusters (GPCs) 250, and one or more partition units 280. The PPU 200 may be connected to a host processor or other PPUs 200 via one or more high-speed NVLink 210 interconnect. The PPU 200 may be connected to a host processor or other peripheral devices via an interconnect 202. The PPU 200 may also be connected to a local memory comprising a number of memory devices 204. In an embodiment, the local memory may comprise a number of dynamic random access memory (DRAM) devices. The DRAM devices may be configured as

a high-bandwidth memory (HBM) subsystem, with multiple DRAM dies stacked within each device.

[0036] The NVLink 210 interconnect enables systems to scale and include one or more PPUs 200 combined with one or more CPUs, supports cache coherence between the PPUs 200 and CPUs, and CPU mastering. Data and/or commands may be transmitted by the NVLink 210 through the hub 230 to/from other units of the PPU 200 such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). The NVLink 210 is described in more detail in conjunction with FIG. 4B.

[0037] The I/O unit 205 is configured to transmit and receive communications (i.e., commands, data, etc.) from a host processor (not shown) over the interconnect 202. The I/O unit 205 may communicate with the host processor directly via the interconnect 202 or through one or more intermediate devices such as a memory bridge. In an embodiment, the I/O unit 205 may communicate with one or more other processors, such as one or more the PPUs 200 via the interconnect 202. In an embodiment, the I/O unit 205 implements a Peripheral Component Interconnect Express (PCIe) interface for communications over a PCIe bus and the interconnect 202 is a PCIe bus. In alternative embodiments, the I/O unit 205 may implement other types of well-known interfaces for communicating with external devices.

[0038] The I/O unit 205 decodes packets received via the interconnect 202. In an embodiment, the packets represent commands configured to cause the PPU 200 to perform various operations. The I/O unit 205 transmits the decoded commands to various other units of the PPU 200 as the commands may specify. For example, some commands may be transmitted to the front end unit 215. Other commands may be transmitted to the hub 230 or other units of the PPU 200 such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). In other words, the I/O unit 205 is configured to route communications between and among the various logical units of the PPU 200.

[0039] In an embodiment, a program executed by the host processor encodes a command stream in a buffer that provides workloads to the PPU 200 for processing. A workload may comprise several instructions and data to be processed by those instructions. The buffer is a region in a memory that is accessible (i.e., read/write) by both the host processor and the PPU 200. For example, the I/O unit 205 may be configured to access the buffer in a system memory connected to the interconnect 202 via memory requests transmitted over the interconnect 202. In an embodiment, the host processor writes the command stream to the buffer and then transmits a pointer to the start of the command stream to the PPU 200. The front end unit 215 receives pointers to one or more command streams. The front end unit 215 manages the one or more streams, reading commands from the streams and forwarding commands to the various units of the PPU 200.

[0040] The front end unit 215 is coupled to a scheduler unit 220 that configures the various GPCs 250 to process tasks defined by the one or more streams. The scheduler unit 220 is configured to track state information related to the various tasks managed by the scheduler unit 220. The state may indicate which GPC 250 a task is assigned to, whether the task is active or inactive, a priority level associated with

the task, and so forth. The scheduler unit 220 manages the execution of a plurality of tasks on the one or more GPCs 250.

[0041] The scheduler unit 220 is coupled to a work distribution unit 225 that is configured to dispatch tasks for execution on the GPCs 250. The work distribution unit 225 may track a number of scheduled tasks received from the scheduler unit 220. In an embodiment, the work distribution unit 225 manages a pending task pool and an active task pool for each of the GPCs 250. The pending task pool may comprise a number of slots (e.g., 32 slots) that contain tasks assigned to be processed by a particular GPC 250. The active task pool may comprise a number of slots (e.g., 4 slots) for tasks that are actively being processed by the GPCs 250. As a GPC 250 finishes the execution of a task, that task is evicted from the active task pool for the GPC 250 and one of the other tasks from the pending task pool is selected and scheduled for execution on the GPC 250. If an active task has been idle on the GPC 250, such as while waiting for a data dependency to be resolved, then the active task may be evicted from the GPC 250 and returned to the pending task pool while another task in the pending task pool is selected and scheduled for execution on the GPC 250.

[0042] The work distribution unit 225 communicates with the one or more GPCs 250 via XBar 270. The XBar 270 is an interconnect network that couples many of the units of the PPU 200 to other units of the PPU 200. For example, the XBar 270 may be configured to couple the work distribution unit 225 to a particular GPC 250. Although not shown explicitly, one or more other units of the PPU 200 may also be connected to the XBar 270 via the hub 230.

[0043] The tasks are managed by the scheduler unit 220 and dispatched to a GPC 250 by the work distribution unit 225. The GPC 250 is configured to process the task and generate results. The results may be consumed by other tasks within the GPC 250, routed to a different GPC 250 via the XBar 270, or stored in the memory 204. The results can be written to the memory 204 via the partition units 280, which implement a memory interface for reading and writing data to/from the memory 204. The results can be transmitted to another PPU 200 or CPU via the NVLink 210. In an embodiment, the PPU 200 includes a number U of partition units 280 that is equal to the number of separate and distinct memory devices 204 coupled to the PPU 200. A partition unit 280 will be described in more detail below in conjunction with FIG. 3B.

[0044] In an embodiment, a host processor executes a driver kernel that implements an application programming interface (API) that enables one or more applications executing on the host processor to schedule operations for execution on the PPU 200. In an embodiment, multiple compute applications are simultaneously executed by the PPU 200 and the PPU 200 provides isolation, quality of service (QOS), and independent address spaces for the multiple compute applications. An application may generate instructions (i.e., API calls) that cause the driver kernel to generate one or more tasks for execution by the PPU 200. The driver kernel outputs tasks to one or more streams being processed by the PPU 200. Each task may comprise one or more groups of related threads, referred to herein as a warp. In an embodiment, a warp comprises 32 related threads that may be executed in parallel. Cooperating threads may refer to a plurality of threads including instructions to perform the task

and that may exchange data through shared memory. Threads and cooperating threads are described in more detail in conjunction with FIG. 4A.

[0045] FIG. 3A illustrates a GPC 250 of the PPU 200 of FIG. 2, in accordance with an embodiment. As shown in FIG. 3A, each GPC 250 includes a number of hardware units for processing tasks. In an embodiment, each GPC 250 includes a pipeline manager 310, a pre-raster operations unit (PROP) 315, a raster engine 325, a work distribution crossbar (WDX) 380, a memory management unit (MMU) 390, and one or more Data Processing Clusters (DPCs) 320. It will be appreciated that the GPC 250 of FIG. 3A may include other hardware units in lieu of or in addition to the units shown in FIG. 3A.

[0046] In an embodiment, the operation of the GPC 250 is controlled by the pipeline manager 310. The pipeline manager 310 manages the configuration of the one or more DPCs 320 for processing tasks allocated to the GPC 250. In an embodiment, the pipeline manager 310 may configure at least one of the one or more DPCs 320 to implement at least a portion of a graphics rendering pipeline. For example, a DPC 320 may be configured to execute a vertex shader program on the programmable streaming multiprocessor (SM) 340. The pipeline manager 310 may also be configured to route packets received from the work distribution unit 225 to the appropriate logical units within the GPC 250. For example, some packets may be routed to fixed function hardware units in the PROP 315 and/or raster engine 325 while other packets may be routed to the DPCs 320 for processing by the primitive engine 335 or the SM 340. In an embodiment, the pipeline manager 310 may configure at least one of the one or more DPCs 320 to implement a neural network model and/or a computing pipeline.

[0047] The PROP unit 315 is configured to route data generated by the raster engine 325 and the DPCs 320 to a Raster Operations (ROP) unit, described in more detail in conjunction with FIG. 3B. The PROP unit 315 may also be configured to perform optimizations for color blending, organize pixel data, perform address translations, and the like.

[0048] The raster engine 325 includes a number of fixed function hardware units configured to perform various raster operations. In an embodiment, the raster engine 325 includes a setup engine, a coarse raster engine, a culling engine, a clipping engine, a fine raster engine, and a tile coalescing engine. The setup engine receives transformed vertices and generates plane equations associated with the geometric primitive defined by the vertices. The plane equations are transmitted to the coarse raster engine to generate coverage information (e.g., an x,y coverage mask for a tile) for the primitive. The output of the coarse raster engine is transmitted to the culling engine where fragments associated with the primitive that fail a z-test are culled, and transmitted to a clipping engine where fragments lying outside a viewing frustum are clipped. Those fragments that survive clipping and culling may be passed to the fine raster engine to generate attributes for the pixel fragments based on the plane equations generated by the setup engine. The output of the raster engine 325 comprises fragments to be processed, for example, by a fragment shader implemented within a DPC 320.

[0049] Each DPC 320 included in the GPC 250 includes an M-Pipe Controller (MPC) 330, a primitive engine 335, and one or more SMs 340. The MPC 330 controls the

operation of the DPC 320, routing packets received from the pipeline manager 310 to the appropriate units in the DPC 320. For example, packets associated with a vertex may be routed to the primitive engine 335, which is configured to fetch vertex attributes associated with the vertex from the memory 204. In contrast, packets associated with a shader program may be transmitted to the SM 340.

[0050] The SM 340 comprises a programmable streaming processor that is configured to process tasks represented by a number of threads. Each SM 340 is multi-threaded and configured to execute a plurality of threads (e.g., 32 threads) from a particular group of threads concurrently. In an embodiment, the SM 340 implements a SIMD (Single-Instruction, Multiple-Data) architecture where each thread in a group of threads (i.e., a warp) is configured to process a different set of data based on the same set of instructions. All threads in the group of threads execute the same instructions. In another embodiment, the SM 340 implements a SIMT (Single-Instruction, Multiple Thread) architecture where each thread in a group of threads is configured to process a different set of data based on the same set of instructions, but where individual threads in the group of threads are allowed to diverge during execution. In an embodiment, a program counter, call stack, and execution state is maintained for each warp, enabling concurrency between warps and serial execution within warps when threads within the warp diverge. In another embodiment, a program counter, call stack, and execution state is maintained for each individual thread, enabling equal concurrency between all threads, within and between warps. When execution state is maintained for each individual thread, threads executing the same instructions may be converged and executed in parallel for maximum efficiency. The SM 340 will be described in more detail below in conjunction with FIG. 4A.

[0051] The MMU 390 provides an interface between the GPC 250 and the partition unit 280. The MMU 390 may provide translation of virtual addresses into physical addresses, memory protection, and arbitration of memory requests. In an embodiment, the MMU 390 provides one or more translation lookaside buffers (TLBs) for performing translation of virtual addresses into physical addresses in the memory 204.

[0052] FIG. 3B illustrates a memory partition unit 280 of the PPU 200 of FIG. 2, in accordance with an embodiment. As shown in FIG. 3B, the memory partition unit 280 includes a Raster Operations (ROP) unit 350, a level two (L2) cache 360, and a memory interface 370. The memory interface 370 is coupled to the memory 204. Memory interface 370 may implement 32, 64, 128, 1024-bit data buses, or the like, for high-speed data transfer. In an embodiment, the PPU 200 incorporates U memory interfaces 370, one memory interface 370 per pair of partition units 280, where each pair of partition units 280 is connected to a corresponding memory device 204. For example, PPU 200 may be connected to up to Y memory devices 204, such as high bandwidth memory stacks or graphics double-data-rate, version 5, synchronous dynamic random access memory, or other types of persistent storage.

[0053] In an embodiment, the memory interface 370 implements an HBM2 memory interface and Y equals half U. In an embodiment, the HBM2 memory stacks are located on the same physical package as the PPU 200, providing substantial power and area savings compared with conven-

tional GDDR5 SDRAM systems. In an embodiment, each HBM2 stack includes four memory dies and Y equals 4, with HBM2 stack including two 128-bit channels per die for a total of 8 channels and a data bus width of 1024 bits.

[0054] In an embodiment, the memory 204 supports Single-Error Correcting Double-Error Detecting (SECCDED) Error Correction Code (ECC) to protect data. ECC provides higher reliability for compute applications that are sensitive to data corruption. Reliability is especially important in large-scale cluster computing environments where PPUs 200 process very large datasets and/or run applications for extended periods.

[0055] In an embodiment, the PPU 200 implements a multi-level memory hierarchy. In an embodiment, the memory partition unit 280 supports a unified memory to provide a single unified virtual address space for CPU and PPU 200 memory, enabling data sharing between virtual memory systems. In an embodiment the frequency of accesses by a PPU 200 to memory located on other processors is traced to ensure that memory pages are moved to the physical memory of the PPU 200 that is accessing the pages more frequently. In an embodiment, the NVLink 210 supports address translation services allowing the PPU 200 to directly access a CPU's page tables and providing full access to CPU memory by the PPU 200.

[0056] In an embodiment, copy engines transfer data between multiple PPUs 200 or between PPUs 200 and CPUs. The copy engines can generate page faults for addresses that are not mapped into the page tables. The memory partition unit 280 can then service the page faults, mapping the addresses into the page table, after which the copy engine can perform the transfer. In a conventional system, memory is pinned (i.e., non-pageable) for multiple copy engine operations between multiple processors, substantially reducing the available memory. With hardware page faulting, addresses can be passed to the copy engines without worrying if the memory pages are resident, and the copy process is transparent.

[0057] Data from the memory 204 or other system memory may be fetched by the memory partition unit 280 and stored in the L2 cache 360, which is located on-chip and is shared between the various GPCs 250. As shown, each memory partition unit 280 includes a portion of the L2 cache 360 associated with a corresponding memory device 204. Lower level caches may then be implemented in various units within the GPCs 250. For example, each of the SMs 340 may implement a level one (L1) cache. The L1 cache is private memory that is dedicated to a particular SM 340. Data from the L2 cache 360 may be fetched and stored in each of the L1 caches for processing in the functional units of the SMs 340. The L2 cache 360 is coupled to the memory interface 370 and the XBar 270.

[0058] The ROP unit 350 performs graphics raster operations related to pixel color, such as color compression, pixel blending, and the like. The ROP unit 350 also implements depth testing in conjunction with the raster engine 325, receiving a depth for a sample location associated with a pixel fragment from the culling engine of the raster engine 325. The depth is tested against a corresponding depth in a depth buffer for a sample location associated with the fragment. If the fragment passes the depth test for the sample location, then the ROP unit 350 updates the depth buffer and transmits a result of the depth test to the raster engine 325. It will be appreciated that the number of partition units 280

may be different than the number of GPCs 250 and, therefore, each ROP unit 350 may be coupled to each of the GPCs 250. The ROP unit 350 tracks packets received from the different GPCs 250 and determines which GPC 250 that a result generated by the ROP unit 350 is routed to through the Xbar 270. Although the ROP unit 350 is included within the memory partition unit 280 in FIG. 3B, in other embodiment, the ROP unit 350 may be outside of the memory partition unit 280. For example, the ROP unit 350 may reside in the GPC 250 or another unit.

[0059] FIG. 4A illustrates the streaming multi-processor 340 of FIG. 3A, in accordance with an embodiment. As shown in FIG. 4A, the SM 340 includes an instruction cache 405, one or more scheduler units 410(K), a register file 420, one or more processing cores 450, one or more special function units (SFUs) 452, one or more load/store units (LSUs) 454, an interconnect network 480, a shared memory/L1 cache 470.

[0060] As described above, the work distribution unit 225 dispatches tasks for execution on the GPCs 250 of the PPU 200. The tasks are allocated to a particular DPC 320 within a GPC 250 and, if the task is associated with a shader program, the task may be allocated to an SM 340. The scheduler unit 410(K) receives the tasks from the work distribution unit 225 and manages instruction scheduling for one or more thread blocks assigned to the SM 340. The scheduler unit 410(K) schedules thread blocks for execution as warps of parallel threads, where each thread block is allocated at least one warp. In an embodiment, each warp executes 32 threads. The scheduler unit 410(K) may manage a plurality of different thread blocks, allocating the warps to the different thread blocks and then dispatching instructions from the plurality of different cooperative groups to the various functional units (i.e., cores 450, SFUs 452, and LSUs 454) during each clock cycle.

[0061] Cooperative Groups is a programming model for organizing groups of communicating threads that allows developers to express the granularity at which threads are communicating, enabling the expression of richer, more efficient parallel decompositions. Cooperative launch APIs support synchronization amongst thread blocks for the execution of parallel algorithms. Conventional programming models provide a single, simple construct for synchronizing cooperating threads: a barrier across all threads of a thread block (i.e., the `syncthreads()` function). However, programmers would often like to define groups of threads at smaller than thread block granularities and synchronize within the defined groups to enable greater performance, design flexibility, and software reuse in the form of collective group-wide function interfaces.

[0062] Cooperative Groups enables programmers to define groups of threads explicitly at sub-block (i.e., as small as a single thread) and multi-block granularities, and to perform collective operations such as synchronization on the threads in a cooperative group. The programming model supports clean composition across software boundaries, so that libraries and utility functions can synchronize safely within their local context without having to make assumptions about convergence. Cooperative Groups primitives enable new patterns of cooperative parallelism, including producer-consumer parallelism, opportunistic parallelism, and global synchronization across an entire grid of thread blocks.

[0063] A dispatch unit 415 is configured to transmit instructions to one or more of the functional units. In the embodiment, the scheduler unit 410(K) includes two dispatch units 415 that enable two different instructions from the same warp to be dispatched during each clock cycle. In alternative embodiments, each scheduler unit 410(K) may include a single dispatch unit 415 or additional dispatch units 415.

[0064] Each SM 340 includes a register file 420 that provides a set of registers for the functional units of the SM 340. In an embodiment, the register file 420 is divided between each of the functional units such that each functional unit is allocated a dedicated portion of the register file 420. In another embodiment, the register file 420 is divided between the different warps being executed by the SM 340. The register file 420 provides temporary storage for operands connected to the data paths of the functional units.

[0065] Each SM 340 comprises L processing cores 450. In an embodiment, the SM 340 includes a large number (e.g., 128, etc.) of distinct processing cores 450. Each core 450 may include a fully-pipelined, single-precision, double-precision, and/or mixed precision processing unit that includes a floating point arithmetic logic unit and an integer arithmetic logic unit. In an embodiment, the floating point arithmetic logic units implement the IEEE 754-2008 standard for floating point arithmetic. In an embodiment, the cores 450 include 64 single-precision (32-bit) floating point cores, 64 integer cores, 32 double-precision (64-bit) floating point cores, and 8 tensor cores.

[0066] Tensor cores configured to perform matrix operations, and, in an embodiment, one or more tensor cores are included in the cores 450. In particular, the tensor cores are configured to perform deep learning matrix arithmetic, such as convolution operations for neural network training and inferring. In an embodiment, each tensor core operates on a 4x4 matrix and performs a matrix multiply and accumulate operation $D=A \times B + C$, where A, B, C, and D are 4x4 matrices.

[0067] In an embodiment, the matrix multiply inputs A and B are 16-bit floating point matrices, while the accumulation matrices C and D may be 16-bit floating point or 32-bit floating point matrices. Tensor Cores operate on 16-bit floating point input data with 32-bit floating point accumulation. The 16-bit floating point multiply requires 64 operations and results in a full precision product that is then accumulated using 32-bit floating point addition with the other intermediate products for a 4x4x4 matrix multiply. In practice, Tensor Cores are used to perform much larger two-dimensional or higher dimensional matrix operations, built up from these smaller elements. An API, such as CUDA 9 C++ API, exposes specialized matrix load, matrix multiply and accumulate, and matrix store operations to efficiently use Tensor Cores from a CUDA-C++ program. At the CUDA level, the warp-level interface assumes 16x16 size matrices spanning all 32 threads of the warp.

[0068] Each SM 340 also comprises M SFUs 452 that perform special functions (e.g., attribute evaluation, reciprocal square root, and the like). In an embodiment, the SFUs 452 may include a tree traversal unit configured to traverse a hierarchical tree data structure. In an embodiment, the SFUs 452 may include texture unit configured to perform texture map filtering operations. In an embodiment, the texture units are configured to load texture maps (e.g., a 2D array of texels) from the memory 204 and sample the texture

maps to produce sampled texture values for use in shader programs executed by the SM 340. In an embodiment, the texture maps are stored in the shared memory/L1 cache 370. The texture units implement texture operations such as filtering operations using mip-maps (i.e., texture maps of varying levels of detail). In an embodiment, each SM 240 includes two texture units.

[0069] Each SM 340 also comprises N LSUs 454 that implement load and store operations between the shared memory/L1 cache 470 and the register file 420. Each SM 340 includes an interconnect network 480 that connects each of the functional units to the register file 420 and the LSU 454 to the register file 420, shared memory/L1 cache 470. In an embodiment, the interconnect network 480 is a crossbar that can be configured to connect any of the functional units to any of the registers in the register file 420 and connect the LSUs 454 to the register file and memory locations in shared memory/L1 cache 470.

[0070] The shared memory/L1 cache 470 is an array of on-chip memory that allows for data storage and communication between the SM 340 and the primitive engine 335 and between threads in the SM 340. In an embodiment, the shared memory/L1 cache 470 comprises 128 KB of storage capacity and is in the path from the SM 340 to the partition unit 280. The shared memory/L1 cache 470 can be used to cache reads and writes. One or more of the shared memory/L1 cache 470, L2 cache 360, and memory 204 are backing stores.

[0071] Combining data cache and shared memory functionality into a single memory block provides the best overall performance for both types of memory accesses. The capacity is usable as a cache by programs that do not use shared memory. For example, if shared memory is configured to use half of the capacity, texture and load/store operations can use the remaining capacity. Integration within the shared memory/L1 cache 470 enables the shared memory/L1 cache 470 to function as a high-throughput conduit for streaming data while simultaneously providing high-bandwidth and low-latency access to frequently reused data.

[0072] When configured for general purpose parallel computation, a simpler configuration can be used compared with graphics processing. Specifically, the fixed function graphics processing units shown in FIG. 2, are bypassed, creating a much simpler programming model. In the general purpose parallel computation configuration, the work distribution unit 225 assigns and distributes blocks of threads directly to the DPCs 320. The threads in a block execute the same program, using a unique thread ID in the calculation to ensure each thread generates unique results, using the SM 340 to execute the program and perform calculations, shared memory/L1 cache 470 to communicate between threads, and the LSU 454 to read and write global memory through the shared memory/L1 cache 470 and the memory partition unit 280. When configured for general purpose parallel computation, the SM 340 can also write commands that the scheduler unit 220 can use to launch new work on the DPCs 320.

[0073] The PPU 200 may be included in a desktop computer, a laptop computer, a tablet computer, servers, supercomputers, a smart-phone (e.g., a wireless, hand-held device), personal digital assistant (PDA), a digital camera, a vehicle, a head mounted display, a hand-held electronic device, and the like. In an embodiment, the PPU 200 is

embodied on a single semiconductor substrate. In another embodiment, the PPU 200 is included in a system-on-a-chip (SoC) along with one or more other devices such as additional PPUs 200, the memory 204, a reduced instruction set computer (RISC) CPU, a memory management unit (MMU), a digital-to-analog converter (DAC), and the like. [0074] In an embodiment, the PPU 200 may be included on a graphics card that includes one or more memory devices 204. The graphics card may be configured to interface with a PCIe slot on a motherboard of a desktop computer. In yet another embodiment, the PPU 200 may be an integrated graphics processing unit (iGPU) or parallel processor included in the chipset of the motherboard.

Exemplary Computing System

[0075] Systems with multiple GPUs and CPUs are used in a variety of industries as developers expose and leverage more parallelism in applications such as artificial intelligence computing. High-performance GPU-accelerated systems with tens to many thousands of compute nodes are deployed in data centers, research facilities, and supercomputers to solve ever larger problems. As the number of processing devices within the high-performance systems increases, the communication and data transfer mechanisms need to scale to support the increased bandwidth.

[0076] FIG. 4B is a conceptual diagram of a processing system 400 implemented using the PPU 200 of FIG. 2, in accordance with an embodiment. The exemplary system 465 may be configured to implement the method 100 shown in FIG. 1. The processing system 400 includes a CPU 430, switch 410, and multiple PPUs 200 each and respective memories 204. The NVLink 210 provides high-speed communication links between each of the PPUs 200. Although a particular number of NVLink 210 and interconnect 202 connections are illustrated in FIG. 4B, the number of connections to each PPU 200 and the CPU 430 may vary. The switch 410 interfaces between the interconnect 202 and the CPU 430. The PPUs 200, memories 204, and NVLinks 210 may be situated on a single semiconductor platform to form a parallel processing module 425. In an embodiment, the switch 410 supports two or more protocols to interface between various different connections and/or links.

[0077] In another embodiment (not shown), the NVLink 210 provides one or more high-speed communication links between each of the PPUs 200 and the CPU 430 and the switch 410 interfaces between the interconnect 202 and each of the PPUs 200. The PPUs 200, memories 204, and interconnect 202 may be situated on a single semiconductor platform to form a parallel processing module 425. In yet another embodiment (not shown), the interconnect 202 provides one or more communication links between each of the PPUs 200 and the CPU 430 and the switch 410 interfaces between each of the PPUs 200 using the NVLink 210 to provide one or more high-speed communication links between the PPUs 200. In another embodiment (not shown), the NVLink 210 provides one or more high-speed communication links between the PPUs 200 and the CPU 430 through the switch 410. In yet another embodiment (not shown), the interconnect 202 provides one or more communication links between each of the PPUs 200 directly. One or more of the NVLink 210 high-speed communication links may be implemented as a physical NVLink interconnect or either an on-chip or on-die interconnect using the same protocol as the NVLink 210.

[0078] In the context of the present description, a single semiconductor platform may refer to a sole unitary semiconductor-based integrated circuit fabricated on a die or chip. It should be noted that the term single semiconductor platform may also refer to multi-chip modules with increased connectivity which simulate on-chip operation and make substantial improvements over utilizing a conventional bus implementation. Of course, the various circuits or devices may also be situated separately or in various combinations of semiconductor platforms per the desires of the user. Alternately, the parallel processing module **425** may be implemented as a circuit board substrate and each of the PPUs **200** and/or memories **204** may be packaged devices. In an embodiment, the CPU **430**, switch **410**, and the parallel processing module **425** are situated on a single semiconductor platform.

[0079] In an embodiment, the signaling rate of each NVLink **210** is 20 to 25 Gigabits/second and each PPU **200** includes six NVLink **210** interfaces (as shown in FIG. 4B, five NVLink **210** interfaces are included for each PPU **200**). Each NVLink **210** provides a data transfer rate of 25 Gigabytes/second in each direction, with six links providing 300 Gigabytes/second. The NVLinks **210** can be used exclusively for PPU-to-PPU communication as shown in FIG. 4B, or some combination of PPU-to-PPU and PPU-to-CPU, when the CPU **430** also includes one or more NVLink **210** interfaces.

[0080] In an embodiment, the NVLink **210** allows direct load/store/atomic access from the CPU **430** to each PPU's **200** memory **204**. In an embodiment, the NVLink **210** supports coherency operations, allowing data read from the memories **204** to be stored in the cache hierarchy of the CPU **430**, reducing cache access latency for the CPU **430**. In an embodiment, the NVLink **210** includes support for Address Translation Services (ATS), allowing the PPU **200** to directly access page tables within the CPU **430**. One or more of the NVLinks **210** may also be configured to operate in a low-power mode.

[0081] FIG. 4C illustrates an exemplary system **465** in which the various architecture and/or functionality of the various previous embodiments may be implemented. The exemplary system **465** may be configured to implement the method **100** shown in FIG. 1.

[0082] As shown, a system **465** is provided including at least one central processing unit **430** that is connected to a communication bus **475**. The communication bus **475** may be implemented using any suitable protocol, such as PCI (Peripheral Component Interconnect), PCI-Express, AGP (Accelerated Graphics Port), HyperTransport, or any other bus or point-to-point communication protocol(s). The system **465** also includes a main memory **440**. Control logic (software) and data are stored in the main memory **440** which may take the form of random access memory (RAM).

[0083] The system **465** also includes input devices **460**, the parallel processing system **425**, and display devices **445**, i.e. a conventional CRT (cathode ray tube), LCD (liquid crystal display), LED (light emitting diode), plasma display or the like. User input may be received from the input devices **460**, e.g., keyboard, mouse, touchpad, microphone, and the like. Each of the foregoing modules and/or devices may even be situated on a single semiconductor platform to form the system **465**. Alternately, the various modules may also be situated separately or in various combinations of semiconductor platforms per the desires of the user.

[0084] Further, the system **465** may be coupled to a network (e.g., a telecommunications network, local area network (LAN), wireless network, wide area network (WAN) such as the Internet, peer-to-peer network, cable network, or the like) through a network interface **435** for communication purposes.

[0085] The system **465** may also include a secondary storage (not shown). The secondary storage includes, for example, a hard disk drive and/or a removable storage drive, representing a floppy disk drive, a magnetic tape drive, a compact disk drive, digital versatile disk (DVD) drive, recording device, universal serial bus (USB) flash memory. The removable storage drive reads from and/or writes to a removable storage unit in a well-known manner.

[0086] Computer programs, or computer control logic algorithms, may be stored in the main memory **440** and/or the secondary storage. Such computer programs, when executed, enable the system **465** to perform various functions. The memory **440**, the storage, and/or any other storage are possible examples of computer-readable media.

[0087] The architecture and/or functionality of the various previous figures may be implemented in the context of a general computer system, a circuit board system, a game console system dedicated for entertainment purposes, an application-specific system, and/or any other desired system. For example, the system **465** may take the form of a desktop computer, a laptop computer, a tablet computer, servers, supercomputers, a smart-phone (e.g., a wireless, hand-held device), personal digital assistant (PDA), a digital camera, a vehicle, a head mounted display, a hand-held electronic device, a mobile phone device, a television, workstation, game consoles, embedded system, and/or any other type of logic.

[0088] While various embodiments have been described above, it should be understood that they have been presented by way of example only, and not limitation. Thus, the breadth and scope of a preferred embodiment should not be limited by any of the above-described exemplary embodiments, but should be defined only in accordance with the following claims and their equivalents.

Graphics Processing Pipeline

[0089] In an embodiment, the PPU **200** comprises a graphics processing unit (GPU). The PPU **200** is configured to receive commands that specify shader programs for processing graphics data. Graphics data may be defined as a set of primitives such as points, lines, triangles, quads, triangle strips, and the like. Typically, a primitive includes data that specifies a number of vertices for the primitive (e.g., in a model-space coordinate system) as well as attributes associated with each vertex of the primitive. The PPU **200** can be configured to process the graphics primitives to generate a frame buffer (i.e., pixel data for each of the pixels of the display).

[0090] An application writes model data for a scene (i.e., a collection of vertices and attributes) to a memory such as a system memory or memory **204**. The model data defines each of the objects that may be visible on a display. The application then makes an API call to the driver kernel that requests the model data to be rendered and displayed. The driver kernel reads the model data and writes commands to the one or more streams to perform operations to process the model data. The commands may reference different shader programs to be implemented on the SMs **340** of the PPU **200**.

including one or more of a vertex shader, hull shader, domain shader, geometry shader, and a pixel shader. For example, one or more of the SMs 340 may be configured to execute a vertex shader program that processes a number of vertices defined by the model data. In an embodiment, the different SMs 340 may be configured to execute different shader programs concurrently. For example, a first subset of SMs 340 may be configured to execute a vertex shader program while a second subset of SMs 340 may be configured to execute a pixel shader program. The first subset of SMs 340 processes vertex data to produce processed vertex data and writes the processed vertex data to the L2 cache 360 and/or the memory 204. After the processed vertex data is rasterized (i.e., transformed from three-dimensional data into two-dimensional data in screen space) to produce fragment data, the second subset of SMs 340 executes a pixel shader to produce processed fragment data, which is then blended with other processed fragment data and written to the frame buffer in memory 204. The vertex shader program and pixel shader program may execute concurrently, processing different data from the same scene in a pipelined fashion until all of the model data for the scene has been rendered to the frame buffer. Then, the contents of the frame buffer are transmitted to a display controller for display on a display device.

[0091] FIG. 5 is a conceptual diagram of a graphics processing pipeline 500 implemented by the PPU 200 of FIG. 2, in accordance with an embodiment. The graphics processing pipeline 500 is an abstract flow diagram of the processing steps implemented to generate 2D computer-generated images from 3D geometry data. As is well-known, pipeline architectures may perform long latency operations more efficiently by splitting up the operation into a plurality of stages, where the output of each stage is coupled to the input of the next successive stage. Thus, the graphics processing pipeline 500 receives input data 501 that is transmitted from one stage to the next stage of the graphics processing pipeline 500 to generate output data 502. In an embodiment, the graphics processing pipeline 500 may represent a graphics processing pipeline defined by the OpenGL® API. As an option, the graphics processing pipeline 500 may be implemented in the context of the functionality and architecture of the previous Figures and/or any subsequent Figure(s).

[0092] As shown in FIG. 5, the graphics processing pipeline 500 comprises a pipeline architecture that includes a number of stages. The stages include, but are not limited to, a data assembly stage 510, a vertex shading stage 520, a primitive assembly stage 530, a geometry shading stage 540, a viewport scale, cull, and clip (VSCC) stage 550, a rasterization stage 560, a fragment shading stage 570, and a raster operations stage 580. In an embodiment, the input data 501 comprises commands that configure the processing units to implement the stages of the graphics processing pipeline 500 and geometric primitives (e.g., points, lines, triangles, quads, triangle strips or fans, etc.) to be processed by the stages. The output data 502 may comprise pixel data (i.e., color data) that is copied into a frame buffer or other type of surface data structure in a memory.

[0093] The data assembly stage 510 receives the input data 501 that specifies vertex data for high-order surfaces, primitives, or the like. The data assembly stage 510 collects the vertex data in a temporary storage or queue, such as by receiving a command from the host processor that includes

a pointer to a buffer in memory and reading the vertex data from the buffer. The vertex data is then transmitted to the vertex shading stage 520 for processing.

[0094] The vertex shading stage 520 processes vertex data by performing a set of operations (i.e., a vertex shader or a program) once for each of the vertices. Vertices may be, e.g., specified as a 4-coordinate vector (i.e., $\langle x, y, z, w \rangle$) associated with one or more vertex attributes (e.g., color, texture coordinates, surface normal, etc.). The vertex shading stage 520 may manipulate individual vertex attributes such as position, color, texture coordinates, and the like. In other words, the vertex shading stage 520 performs operations on the vertex coordinates or other vertex attributes associated with a vertex. Such operations commonly including lighting operations (i.e., modifying color attributes for a vertex) and transformation operations (i.e., modifying the coordinate space for a vertex). For example, vertices may be specified using coordinates in an object-coordinate space, which are transformed by multiplying the coordinates by a matrix that translates the coordinates from the object-coordinate space into a world space or a normalized-device-coordinate (NCD) space. The vertex shading stage 520 generates transformed vertex data that is transmitted to the primitive assembly stage 530.

[0095] The primitive assembly stage 530 collects vertices output by the vertex shading stage 520 and groups the vertices into geometric primitives for processing by the geometry shading stage 540. For example, the primitive assembly stage 530 may be configured to group every three consecutive vertices as a geometric primitive (i.e., a triangle) for transmission to the geometry shading stage 540. In some embodiments, specific vertices may be reused for consecutive geometric primitives (e.g., two consecutive triangles in a triangle strip may share two vertices). The primitive assembly stage 530 transmits geometric primitives (i.e., a collection of associated vertices) to the geometry shading stage 540.

[0096] The geometry shading stage 540 processes geometric primitives by performing a set of operations (i.e., a geometry shader or program) on the geometric primitives. Tessellation operations may generate one or more geometric primitives from each geometric primitive. In other words, the geometry shading stage 540 may subdivide each geometric primitive into a finer mesh of two or more geometric primitives for processing by the rest of the graphics processing pipeline 500. The geometry shading stage 540 transmits geometric primitives to the viewport SCC stage 550.

[0097] In an embodiment, the graphics processing pipeline 500 may operate within a streaming multiprocessor and the vertex shading stage 520, the primitive assembly stage 530, the geometry shading stage 540, the fragment shading stage 570, and/or hardware/software associated therewith, may sequentially perform processing operations. Once the sequential processing operations are complete, in an embodiment, the viewport SCC stage 550 may utilize the data. In an embodiment, primitive data processed by one or more of the stages in the graphics processing pipeline 500 may be written to a cache (e.g. L1 cache, a vertex cache, etc.). In this case, in an embodiment, the viewport SCC stage 550 may access the data in the cache. In an embodiment, the viewport SCC stage 550 and the rasterization stage 560 are implemented as fixed function circuitry.

[0098] The viewport SCC stage **550** performs viewport scaling, culling, and clipping of the geometric primitives. Each surface being rendered to is associated with an abstract camera position. The camera position represents a location of a viewer looking at the scene and defines a viewing frustum that encloses the objects of the scene. The viewing frustum may include a viewing plane, a rear plane, and four clipping planes. Any geometric primitive entirely outside of the viewing frustum may be culled (i.e., discarded) because the geometric primitive will not contribute to the final rendered scene. Any geometric primitive that is partially inside the viewing frustum and partially outside the viewing frustum may be clipped (i.e., transformed into a new geometric primitive that is enclosed within the viewing frustum). Furthermore, geometric primitives may each be scaled based on a depth of the viewing frustum. All potentially visible geometric primitives are then transmitted to the rasterization stage **560**.

[0099] The rasterization stage **560** converts the 3D geometric primitives into 2D fragments (e.g. capable of being utilized for display, etc.). The rasterization stage **560** may be configured to utilize the vertices of the geometric primitives to setup a set of plane equations from which various attributes can be interpolated. The rasterization stage **560** may also compute a coverage mask for a plurality of pixels that indicates whether one or more sample locations for the pixel intercept the geometric primitive. In an embodiment, z-testing may also be performed to determine if the geometric primitive is occluded by other geometric primitives that have already been rasterized. The rasterization stage **560** generates fragment data (i.e., interpolated vertex attributes associated with a particular sample location for each covered pixel) that are transmitted to the fragment shading stage **570**.

[0100] The fragment shading stage **570** processes fragment data by performing a set of operations (i.e., a fragment shader or a program) on each of the fragments. The fragment shading stage **570** may generate pixel data (i.e., color values) for the fragment such as by performing lighting operations or sampling texture maps using interpolated texture coordinates for the fragment. The fragment shading stage **570** generates pixel data that is transmitted to the raster operations stage **580**.

[0101] The raster operations stage **580** may perform various operations on the pixel data such as performing alpha tests, stencil tests, and blending the pixel data with other pixel data corresponding to other fragments associated with the pixel. When the raster operations stage **580** has finished processing the pixel data (i.e., the output data **502**), the pixel data may be written to a render target such as a frame buffer, a color buffer, or the like.

[0102] It will be appreciated that one or more additional stages may be included in the graphics processing pipeline **500** in addition to or in lieu of one or more of the stages described above. Various implementations of the abstract graphics processing pipeline may implement different stages. Furthermore, one or more of the stages described above may be excluded from the graphics processing pipeline in some embodiments (such as the geometry shading stage **540**). Other types of graphics processing pipelines are contemplated as being within the scope of the present disclosure. Furthermore, any of the stages of the graphics processing pipeline **500** may be implemented by one or more dedicated hardware units within a graphics processor such as PPU **200**. Other stages of the graphics processing

pipeline **500** may be implemented by programmable hardware units such as the SM **340** of the PPU **200**.

[0103] The graphics processing pipeline **500** may be implemented via an application executed by a host processor, such as a CPU. In an embodiment, a device driver may implement an application programming interface (API) that defines various functions that can be utilized by an application in order to generate graphical data for display. The device driver is a software program that includes a plurality of instructions that control the operation of the PPU **200**. The API provides an abstraction for a programmer that lets a programmer utilize specialized graphics hardware, such as the PPU **200**, to generate the graphical data without requiring the programmer to utilize the specific instruction set for the PPU **200**. The application may include an API call that is routed to the device driver for the PPU **200**. The device driver interprets the API call and performs various operations to respond to the API call. In some instances, the device driver may perform operations by executing instructions on the CPU. In other instances, the device driver may perform operations, at least in part, by launching operations on the PPU **200** utilizing an input/output interface between the CPU and the PPU **200**. In an embodiment, the device driver is configured to implement the graphics processing pipeline **500** utilizing the hardware of the PPU **200**.

[0104] Various programs may be executed within the PPU **200** in order to implement the various stages of the graphics processing pipeline **500**. For example, the device driver may launch a kernel on the PPU **200** to perform the vertex shading stage **520** on one SM **340** (or multiple SMs **340**). The device driver (or the initial kernel executed by the PPU **300**) may also launch other kernels on the PPU **300** to perform other stages of the graphics processing pipeline **500**, such as the geometry shading stage **540** and the fragment shading stage **570**. In addition, some of the stages of the graphics processing pipeline **500** may be implemented on fixed unit hardware such as a rasterizer or a data assembler implemented within the PPU **300**. It will be appreciated that results from one kernel may be processed by one or more intervening fixed function hardware units before being processed by a subsequent kernel on an SM **340**.

Machine Learning

[0105] Deep neural networks (DNNs) developed on processors, such as the PPU **200** have been used for diverse use cases, from self-driving cars to faster drug development, from automatic image captioning in online image databases to smart real-time language translation in video chat applications. Deep learning is a technique that models the neural learning process of the human brain, continually learning, continually getting smarter, and delivering more accurate results more quickly over time. A child is initially taught by an adult to correctly identify and classify various shapes, eventually being able to identify shapes without any coaching. Similarly, a deep learning or neural learning system needs to be trained in object recognition and classification for it get smarter and more efficient at identifying basic objects, occluded objects, etc., while also assigning context to objects.

[0106] At the simplest level, neurons in the human brain look at various inputs that are received, importance levels are assigned to each of these inputs, and output is passed on to other neurons to act upon. An artificial neuron or percep-

tron is the most basic model of a neural network. In one example, a perceptron may receive one or more inputs that represent various features of an object that the perceptron is being trained to recognize and classify, and each of these features is assigned a certain weight based on the importance of that feature in defining the shape of an object.

[0107] A deep neural network (DNN) model includes multiple layers of many connected perceptrons (e.g., nodes) that can be trained with enormous amounts of input data to quickly solve complex problems with high accuracy. In one example, a first layer of the DLL model breaks down an input image of an automobile into various sections and looks for basic patterns such as lines and angles. The second layer assembles the lines to look for higher level patterns such as wheels, windshields, and mirrors. The next layer identifies the type of vehicle, and the final few layers generate a label for the input image, identifying the model of a specific automobile brand.

[0108] Once the DNN is trained, the DNN can be deployed and used to identify and classify objects or patterns in a process known as inference. Examples of inference (the process through which a DNN extracts useful information from a given input) include identifying handwritten numbers on checks deposited into ATM machines, identifying images of friends in photos, delivering movie recommendations to over fifty million users, identifying and classifying different types of automobiles, pedestrians, and road hazards in driverless cars, or translating human speech in real-time.

[0109] During training, data flows through the DNN in a forward propagation phase until a prediction is produced that indicates a label corresponding to the input. If the neural network does not correctly label the input, then errors between the correct label and the predicted label are analyzed, and the weights are adjusted for each feature during a backward propagation phase until the DNN correctly labels the input and other inputs in a training dataset. Training complex neural networks requires massive amounts of parallel computing performance, including floating-point multiplications and additions that are supported by the PPU 200. Inferencing is less compute-intensive than training, being a latency-sensitive process where a trained neural network is applied to new inputs it has not seen before to classify images, translate speech, and generally infer new information.

[0110] Neural networks rely heavily on matrix math operations, and complex multi-layered networks require tremendous amounts of floating-point performance and bandwidth for both efficiency and speed. With thousands of processing cores, optimized for matrix math operations, and delivering tens to hundreds of TFLOPS of performance, the PPU 200 is a computing platform capable of delivering performance required for deep neural network-based artificial intelligence and machine learning applications.

Hybrid Denoised Alpha-Tested Shadows

[0111] Ray-tracing shadows of alpha tested geometry is very expensive. Combining noisy visibility samples from raytraced opaque geometry and shadow maps that only alpha tested geometry is rendered to results in hybrid denoised shadows that show penumbras of high quality and hard ray-traced shadows at contact points with a low performance impact.

[0112] In one embodiment, a hitT value (input needed for the denoiser) is estimated from an occluding texel of a noisy

shadow map lookup. This shadow map only contains alpha-tested primitives. The min of the estimated hitT and the exact hitT obtained from ray-tracing into a BVH that only contains opaque (e.g. non-alpha-tested) geometry is fed into a denoiser to obtain hybrid denoised shadows.

[0113] This approach does not blend between hard shadows and shadow map filtered shadows, but feeds a hybrid signal into a shadow denoiser.

[0114] FIG. 6 illustrates a flowchart of a method 600 for joining opaque and alpha-tested buffers, in accordance with an embodiment. Although method 600 is described in the context of a processing unit, the method 600 may also be performed by a program, custom circuitry, or by a combination of custom circuitry and a program. For example, the method 600 may be executed by a GPU (graphics processing unit), CPU (central processing unit), or any processor capable of performing parallel path space filtering by hashing. Furthermore, persons of ordinary skill in the art will understand that any system that performs method 600 is within the scope and spirit of embodiments of the present invention.

[0115] As shown in operation 602, ray-tracing of opaque geometry in a scene is performed to create an opaque buffer. In one embodiment, the opaque buffer may include visibility and distance values for originating pixels within the scene having rays that intersect with occluding geometry that is not alpha-tested. In another embodiment, rays may originate from originating pixels in the scene, and may be traced from the originating pixels to a light source.

[0116] Additionally, in one embodiment, if a ray originating from a pixel intersects opaque geometry, a visibility value for that pixel may be assigned a value of zero (indicating a hit/intersection with the opaque geometry). In another embodiment, if a ray originating from a pixel does not intersect opaque geometry, the visibility value for that pixel may be assigned a value of one (indicating a miss/no intersection with the opaque geometry). In yet another embodiment, a distance value for a pixel may include a hitT value that may be determined by identifying a distance between the pixel and the intersecting opaque geometry.

[0117] Further, as shown in operation 604, visibility and hitT values of alpha-tested geometry in the scene are computed utilizing shadow map lookups to create an alpha-tested buffer. One exemplary method for performing such shadow map lookups is shown above.

[0118] Further still, as shown in operation 606, the opaque buffer and the alpha-tested buffer are joined to create a single resulting buffer. In one embodiment, the opaque buffer and the alpha-tested buffer may be joined utilizing a min operation.

[0119] Also, as shown in operation 608, the single resulting buffer is passed to a denoiser. In one embodiment, the denoiser may perform one or more denoising operations on the rendered scene, utilizing the single resulting buffer. In another embodiment, a portion of opaque geometry may be processed utilizing shadow map lookups. In yet another embodiment, a portion of alpha-tested geometry may be processed utilizing standard ray-tracing techniques.

[0120] In this way, combining noisy visibility samples from raytraced opaque geometry and shadow maps that only alpha-tested geometry is rendered to may result in hybrid denoised shadows that show penumbras of high quality and hard raytraced shadows at contact points with a low performance impact.

[0121] In yet another embodiment, the creation and joining of the opaque and alpha-tested buffers may be performed utilizing a parallel processing unit (PPU) such as the PPU 200 illustrated in FIG. 2.

[0122] FIG. 7 illustrates a flowchart of a method 700 for implementing denoised shadows using only shadow map lookups, in accordance with an embodiment. Although method 700 is described in the context of a processing unit, the method 700 may also be performed by a program, custom circuitry, or by a combination of custom circuitry and a program. For example, the method 700 may be executed by a GPU (graphics processing unit), CPU (central processing unit), or any processor capable of performing parallel path space filtering by hashing. Furthermore, persons of ordinary skill in the art will understand that any system that performs method 700 is within the scope and spirit of embodiments of the present invention.

[0123] As shown in operation 702, a ray is generated from an originating pixel within a scene being rendered. Additionally, as shown in operation 704, one or more shadow map lookups are performed for the originating pixel to determine a noisy shadow signal. Further, as shown in operation 706, the noisy shadow signal is denoised, utilizing a denoiser.

[0124] In yet another embodiment, rays may be generated, and shadow map lookups may be performed, utilizing a parallel processing unit (PPU) such as the PPU 200 illustrated in FIG. 2.

[0125] In this way, denoised shadows may be obtained utilizing only shadow map lookups.

[0126] FIG. 8 illustrates an exemplary determination 800 of hybrid denoised shadows, in accordance with an embodiment. As shown, a ray r 802 originating at a screen pixel p 804 is traced into a bounding volume hierarchy (BVH) 806 containing opaque geometry 808. The intersection of the ray r 802 with the opaque geometry 808 of the BVH 806 is used to determine initial hitT and visibility values (e.g., hitT0 and visibility0).

[0127] Additionally, the screen pixel p 804 is then transformed into a shadow map space 810, and occluding hitT and visibility values (e.g., hitT1 and visibility1) are estimated from shadow map data. The value for hitT is then determined by performing a min operation (e.g., $\text{hitT} = \min(\text{hitT0}, \text{hitT1})$). The value for visibility is also determined by performing a min operation (e.g., $\text{visibility} = \min(\text{visibility0}, \text{visibility1})$). These determined visibility and hitT values are then denoised to determine a denoised shadow.

[0128] While various embodiments have been described above, it should be understood that they have been presented by way of example only, and not limitation. Thus, the breadth and scope of a preferred embodiment should not be limited by any of the above-described exemplary embodiments, but should be defined only in accordance with the following claims and their equivalents.

What is claimed is:

1. A method comprising:

for at least one pixel included in an image representing a scene:—

determining a distance of the at least one pixel from an illuminant of the scene, and

determining that the at least one pixel depicts an area in a shadow; and

rendering the at least one pixel with an amount of blurring, wherein the amount of blurring corresponds to the distance.

2. The method of claim 1, wherein determining the distance of the at least one pixel from the illuminant is based on a generated ray that simulates a path of a particle of light from the at least one pixel toward the illuminant.

3. The method of claim 1, wherein determining that the at least one pixel is within a shadow is based on a shadow map corresponding to the scene.

4. The method of claim 1, wherein determining that the at least one pixel depicts the area in the shadow is based on determining that a depth of the at least one pixel is larger than a depth of a geometry intersected by a generated ray that simulates a path of a particle of light from the at least one pixel toward the illuminant.

5. The method of claim 1, further comprising projecting a position of the at least one pixel to a two-dimensional (2D) coordinate system of a shadow map for performing a lookup in the shadow map to determine at least one of:

the distance of the at least one pixel from an illuminant of the scene, or

that the at least one pixel depicts the area in the shadow.

6. The method of claim 1, wherein the illuminant is a light source within the scene.

7. The method of claim 1, further comprising determining the amount of blurring using a function that increases the blurring as the distance increases.

8. The method of claim 7, wherein the function increases the blurring as the distance increases to increasingly soften the shadow at a location of the at least one pixel as the distance increases.

9. The method of claim 1, wherein the scene is at least one of:

a scene of a gaming application;

a virtual reality scene; or

an augmented reality scene.

10. A system comprising:

one or more processors to:

determine, for at least one pixel included in an image representing a scene:—

a distance of the at least one pixel from an illuminant of the scene, and

that the at least one pixel depicts an area in a shadow; and

render the at least one pixel with an amount of blurring, wherein the amount of blurring corresponds to the distance.

11. The system of claim 10, wherein the one or more processors are to determine the distance of the at least one pixel from the illuminant based on a generated ray that simulates a path of a particle of light from the at least one pixel toward the illuminant

12. The system of claim 10, wherein the one or more processors are to determine that the at least one pixel is within a shadow based on a shadow map corresponding to the scene.

13. The system of claim 10, wherein the one or more processors are to determine that the at least one pixel depicts the area in the shadow based on determining that a depth of the at least one pixel is larger than a depth of a geometry intersected by a generated ray that simulates a path of a particle of light from the at least one pixel toward the illuminant.

14. The system of claim **10**, wherein the one or more processors are further to project a position of the at least one pixel to a two-dimensional (2D) coordinate system of a shadow map for performing a lookup in the shadow map to determine at least one of:

the distance of the at least one pixel from an illuminant of the scene, or

that the at least one pixel depicts the area in the shadow.

15. The system of claim **10**, wherein the illuminant is a light source within the scene.

16. The system of claim **10**, wherein the one or more processors are further to determine the amount of blurring using a function that increases the blurring as the distance increases.

17. The system of claim **16**, wherein the function increases the blurring as the distance increases to increasingly soften the shadow at a location of the at least one pixel as the distance increases.

18. The system of claim **10**, wherein the scene is at least one of:

a scene of a gaming application;

a virtual reality scene; or

an augmented reality scene.

19. A processor comprising:

circuitry to render, with an amount of blurring, at least one pixel of an image representing a scene, wherein the at least one pixel depicts an area of the scene within a shadow, and the amount of blurring corresponds to a distance of the at least one pixel from an illuminant of the scene.

* * * * *